

1. Dato il linguaggio $A_{TM} = \{(M, w) \mid M \text{ è una mdT e } M \text{ accetta } w\}$, dimostrare che esso è indecidibile; dimostrare che il complemento di A_{TM} non è Turing-riconoscibile (4 punti). **FATTO**

TEOREMA: Il linguaggio $A_{TM} = \{(M, w) \mid M \text{ è una mdT ed } M \text{ accetta } w\}$.

DIMOSTRAZIONE:

Supponiamo per assurdo che A_{TM} sia decidibile. Allora esiste un decisore H per A_{TM} .

Sull'input (M, w) , dove M è una mdT e w è una stringa, H si ferma ed accetta se M accetta w . Insieme, H si ferma e rifiuta se M non accetta w .

Purunque, assumiamo che H sia una mdT, dove $H((M, w)) = \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ non accetta } w \end{cases}$

Costruiamo una mdT D usando H come sottoprocedura. Questa nuova mdT chiama H per determinare cosa fa M quando l'input di H è la sua stessa descrizione (M) . Una volta che D ha determinato questa informazione, essa fa il contrario: rifiuta se M accetta ed accetta se M rifiuta.

Anche nel caso in cui eseguiamo D con la sua stessa descrizione (D) in input, $D((D))$ accetta se D rifiuta (D) e rifiuta se D accetta (D) .

Quindi, indipendentemente da ciò che fa D , essa è costretta a fare il contrario. Questo è una contraddizione. $\square$

COROLLARIO 4.23

$\overline{A_{TM}}$ non è Turing-riconoscibile.

DIMOSTRAZIONE. Sappiamo che A_{TM} è Turing-riconoscibile. Se anche $\overline{A_{TM}}$ fosse Turing-riconoscibile, A_{TM} sarebbe decidibile. Il Teorema 4.11 ci dice che A_{TM} non è decidibile, quindi $\overline{A_{TM}}$ non può essere Turing-riconoscibile.

2. Dimostrare il teorema di Cook-Levine. (4 punti)

Cook-Levin

TEOREMA: SAT è NP-completo.

DIMOSTRAZIONE:

La dimostrazione consiste in due passi principali:

1. mostrare che SAT è in NP;
2. mostrare che ogni linguaggio in NP è riducibile a SAT.

Dato A in NP, sia N la mdT non deterministica che decide A in tempo polinomiale n^k .

Y stringa w costruire una formula booleana $\varphi(A, w)$ t.c. w in A se e solo se $\varphi(A, w)$ è soddisfacibile.

Per rappresentare una computazione di N usiamo un tableau, una tabella $n^k \times n^k$ di configurazioni, dove:

- ogni riga rappresenta una configurazione della macchina;
- la prima riga è la configurazione iniziale di N su input w , e ciascuna riga segue dalla precedente in accordo alla funzione di transizione di N ;
- ogni configurazione inizia e finisce con $\#$.

Quindi il tableau ha un alfabeto Σ composto da quello di N , e insieme degli stati e $\#$.

Un tableau è accettabile se qualche riga è una configurazione accettabile. A questo punto, stabilire se la macchina accetta equivale a chiedersi se esiste un tableau accettabile, ed è questo quello che vogliamo tradurre in formula booleana.

Per ogni $i, j = 1 \dots n^k$, e per ogni s in Σ , sia $x_{i,j,s} = 1$ se la cella (i, j) contiene s , altrimenti $x_{i,j,s} = 0$.

Progettiamo la formula φ in modo che un assegnamento che la soddisfa corrisponda ad un tableau accettabile per N su w : $\varphi(N, w) = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{accept}} \wedge \varphi_{\text{move}}$

φ_{cell} garantisce che ogni cella del tableau contenga esattamente un simbolo.

$$\varphi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^2} \left(\bigvee_{s \in C} (x_{i,j,s}) \wedge \left(\bigwedge_{s_1 \neq s_2 \in C} (\neg (x_{i,j,s_1} \vee x_{i,j,s_2})) \right) \right)$$

Dimensione: $(2n^2 + 3) \times (1 + 1 \cdot C^2) = O(n^4)$

φ_{start} impone che la prima riga del tableau rappresenti la configurazione iniziale di N su w .

$$\varphi_{\text{start}} = x_{1,1,w_1} \wedge x_{1,2,w_2} \wedge x_{1,3,w_3} \wedge \dots \wedge x_{1,n,w_n} \wedge x_{2,1,w_1} \wedge x_{2,2,w_2} \wedge \dots \wedge x_{2,n,w_n} \wedge \dots \wedge x_{n,1,w_1} \wedge x_{n,2,w_2} \wedge \dots \wedge x_{n,n,w_n}$$

Dimensione: $2n^2 + 3 = O(n^2)$

φ_{accept} chiede che in almeno una cella del tableau ci sia lo stato di accettazione.

$$\varphi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^2} x_{i,j,q_{\text{accept}}}$$

Dimensione: $(2n^2 + 3)^2 = O(n^{2n})$

φ_{move} serve per garantire che ogni riga del tableau derivi dalla precedente in modo corretto. Per fare questo, si analizzano tutte le finestre 2×3 del tableau.

Diciamo che una finestra 2×3 è *ok* se non viola le azioni specificate dalla funzione di transizione di N .

Per esempio, siamo a, b, c elementi dell'alfabeto di N , e q_1, q_2 stati di N . Si assuma che quando nello stato q_1 con la testina che legge una a , N scrive una b , resta nello stato q_1 , e muove a destra; e che quando nello stato q_1 con la testina che legge una b , N non deterministicamente:

1. scrive una c , entra in q_2 , e muove a sinistra, oppure
2. scrive una a , entra in q_2 , e muove a destra.

In termini formali, $\delta(q_1, a) = \{(q_1, b, R)\}$ e $\delta(q_1, b) = \{(q_2, c, L), (q_2, a, R)\}$.

Annando alla costruzione di φ_{move} , essa garantisce che tutte le finestre nel tableau sono *ok* (ovvero finestra contiene sei celle).

$$\varphi_{\text{move}} = \bigwedge_{1 \leq i, j \leq n^2} (\text{la finestra } (i, j) \text{ è ok})$$

↳ ha cella (i, j) in posizione centrale in alto

Dimensione di φ_{move} :

Dimensione di una formula per una window *ok*: 6

Numero di possibili windows *ok* per cella: $\max(C)^6$

Numero di possibili celle: $(2n^2 + 3)n^2$

$$\Rightarrow 6 \cdot \max(C)^6 \cdot (2n^2 + 3)n^2 = O(n^{2n})$$

Dimensione di $\varphi(M, w) = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{accept}} \wedge \varphi_{\text{move}}$

$$O(n^{2n}) + O(n^2) + O(n^{2n}) + O(n^{2n}) = O(n^{2n})$$

Allora è costruita in tempo polinomiale. Questo implica che L è riducibile in tempo polinomiale a SAT.

Abbiamo che: $w \in L \Leftrightarrow \varphi(M, w)$ è soddisfacibile. $\square$

3. Dimostrare qual è la complessità temporale e spaziale del linguaggio $\{0^n 1^n, n \geq 0\}$ (4 punti).

9. Dimostrare qual è la complessità temporale e spaziale del linguaggio $\{0^n 1^n, n \geq 0\}$ (4 punti).

La complessità spaziale si riferisce alla quantità di memoria richiesta per eseguire un algoritmo o una MDT in base all'input. Solitamente viene misurata in termini di spazio di lavoro utilizzato, ovvero la quantità di memoria ausiliaria richiesta oltre alla memoria per memorizzare l'input stesso. La complessità spaziale può essere espressa come una funzione di n , dove n è la dimensione dell'input.

La complessità temporale, d'altra parte, si riferisce al numero di operazioni elementari (come istruzioni, passaggi di calcolo, confronti, accessi alla memoria, etc.) eseguite da un algoritmo o da una MDT per completare l'esecuzione. La complessità temporale può anche essere espressa come una funzione di n .

Input: n (dimensione delle stringhe)

CC Pagina 7

Output: Contatore (numero di stringhe generate)

Contatore = 0

Per ogni stringa s di lunghezza $2n$:

Se la stringa s contiene n zeri e n uni:

Incrementa il contatore di 1

Restituisci Contatore

Ora, per quanto riguarda la complessità spaziale e temporale:

Complessità spaziale: La MDT richiede spazio per memorizzare il contatore e la stringa corrente. La lunghezza massima della stringa è $2n$, quindi il requisito di spazio è $O(n)$. Quindi, la complessità spaziale è $O(n)$.

Complessità temporale: La MDT genera tutte le possibili stringhe di lunghezza $2n$. Il numero totale di tali stringhe è $2^{2n} = 4^n$. Quindi, il numero di iterazioni del ciclo è 4^n . Poiché ogni iterazione richiede tempo costante per verificare se la stringa contiene n zeri e n uni, la complessità temporale complessiva è $O(4^n)$ o $O(2^{2n})$. Quindi, la complessità temporale è esponenziale rispetto a n .

In sintesi, la complessità spaziale del linguaggio $L_0 L_1$ è $O(n)$, mentre la complessità temporale è $O(2^{2n})$ o $O(4^n)$, che è esponenziale rispetto a n .

4. Dimostrare qual è la complessità temporale e spaziale della funzione $f(x,y)=|x-y|$ (4 punti).

5. Dimostrare che P è chiusa rispetto a unione, concatenazione e complemento (4 punti).

Dobbiamo dimostrare che se due linguaggi L_1 e L_2 sono in P, allora anche la loro unione, concat, compl, intersez sono in P.

- **Unione:** supponiamo L_1 e L_2 in P. abbiamo quindi due mdt deterministiche che riconoscono in tempo polinomiale. Costruiamo quindi una mdt che simula entrambe le macchine per L_1 e L_2 . La macchina prende in input una stringa e la simula su entrambe le macchine. Se una delle due macchine accetta, allora la mdt principale accetta. Se entrambe rifiutano, allora la principale rifiuta.
- **Concatenazione:** Costruiamo una mdt che simula entrambe le macchine per L_1 e L_2 in modo sequenziale. La macchina principale prende in input una stringa e la simula prima sulla mdt per L_1 . Se accetta, allora simula la stessa stringa sulla macchina per L_2 . Se anche questa accetta, allora la macchina principale accetta. Se una delle due macchine rifiuta, allora la macchina principale rifiuta l'input.
- **Complemento:** Costruiamo una mdt che INVERTE il comportamento della mdt per L . Quindi la macchina principale simula l'input sulla mdt per L . se la mdt accetta, la macchina principale rifiuta, e viceversa.
- **Intersezione:** Costruiamo una macchina che simula le computazioni di M_1 e M_2 su ogni possibile input. Se sia M_1 che M_2 accettano, allora la macchina principale accetta. Se una tra M_1 e M_2 rifiuta, allora la macchina principale rifiuta.

6. Fornire un esempio di linguaggio non Turing-riconoscibile e dimostrare perchè (4 punti).

COROLLARIO: $\overline{A_{TM}}$ non è Turing-riconoscibile.

DIMOSTRAZIONE:

Sappiamo che A_{TM} è Turing-riconoscibile. Se anche $\overline{A_{TM}}$ fosse Turing-riconoscibile, A_{TM} sarebbe decidibile.

Il teorema ci dice che A_{TM} non è decidibile, quindi $\overline{A_{TM}}$ non può essere Turing-riconoscibile.

TEOREMA: Il linguaggio $ATM = \{ \langle M, w \rangle \mid M \text{ è una mdT ed } M \text{ accetta } w \}$.

PROVOSTRAZIONE:

Supponiamo per assurdo che ATM sia decidibile. Allora esiste un decisore H per ATM .

Sull'input $\langle M, w \rangle$, dove M è una mdT e w è una stringa, H si ferma ed accetta se M accetta w . Inoltre, H si ferma e rifiuta se M non accetta w .

Quindi, assumiamo che H sia una mdT, dove $H(\langle M, w \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ non accetta } w \end{cases}$

Costruiamo una mdT D avente H come sottoprocedura. Questa nuova mdT chiama H per determinare cosa fa H quando l'input di H è la sua stessa descrizione $\langle H \rangle$. Una volta che D ha determinato questa informazione, essa fa il contrario: rifiuta se H accetta ed accetta se H rifiuta.

Anche nel caso in cui eseguiamo D con la sua stessa descrizione $\langle D \rangle$ in input, $D(\langle D \rangle)$ accetta se D rifiuta $\langle D \rangle$ e rifiuta se D accetta $\langle D \rangle$.

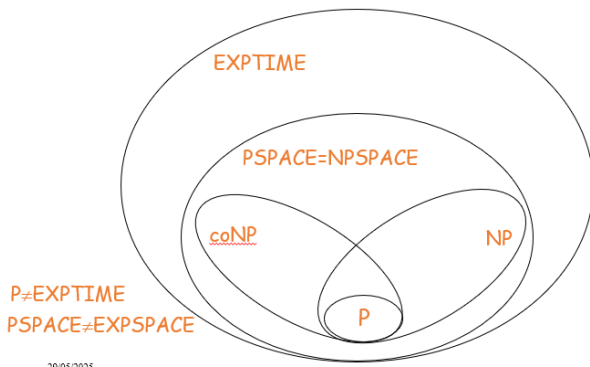
Quindi, indipendentemente da ciò che fa D , essa è costretta a fare il contrario. Questo è una contraddizione. $\square$

7. $ATM = \{ \langle M, w \rangle \mid \text{la mdT } M \text{ accetta } w \}$ è indecidibile. SOPRA

8. Fornire le definizioni delle classi P, NP, co-NP, PSPACE, NPSPACE, EXPTIME; illustrare graficamente il rapporto fra tali classi e spiegare perché (4 punti).

- **P (polynomial time):** la classe P è la classe dei problemi che si possono risolvere da una mdT DETERMINISTICA in tempo polinomiale. In altre parole, un problema è in P se esiste un algoritmo efficiente (polinomiale) in grado di risolverlo.
- **NP (nondeterministic polynomial time):** la classe NP è la classe dei problemi che possono essere verificati in tempo polinomiale da una mdT NON deterministica. In altre parole, un problema è in NP se per ogni istanza del problema è possibile verificare se la soluzione proposta è corretta, senza controllare che la soluzione venga trovata in modo efficiente.
- **coNP (complemento di NP):** la classe co-NP è l'insieme dei complementi dei problemi in NP. Un problema è in co-NP se il suo complemento, ovvero il linguaggio degli input che producono una risposta negativa, può essere verificato in tempo polinomiale.
- **PSPACE (polynomial space):** la classe PSPACE è l'insieme dei problemi che possono essere risolti da una mdT DETERMINISTICA utilizzando una quantità di spazio polinomiale rispetto alla dimensione dell'input. Questa classe comprende problemi che richiedono spazio di memoria SUPERIORE a quello consentito nella classe P.
- **NPSPACE (nondeterministic polynomial space):** è l'equivalente NON DETERMINISTICO della classe PSPACE. Comprende problemi che possono essere verificati in tempo polinomiale da una mdT NON DETERMINISTICA utilizzando una quantità di spazio di memoria polinomiale.
- **EXPTIME (exponential time):** è la classe dei problemi che possono essere risolti da una mdT DETERMINISTICA in tempo esponenziale.

$$P \subseteq NP \subseteq PSPACE \subseteq NPSPACE \subseteq EXPTIME$$



Ogni problema in P è risolvibile anche in NP e così via.

La classe PSPACE è più ampia della classe P, e la classe NPSPACE è più ampia della classe NP.

Questo perché spazio di memoria maggiore consente di risolvere problemi più complessi.

EXPTIME è la più ampia, perché i problemi

richiedono tempo esponenziale per essere risolti.

9. Dimostrare il teorema di Savitch (4 punti).

TEOREMA: Il teorema di Savitch afferma che per ogni funzione $f: \mathbb{N} \rightarrow \mathbb{R}^+$, dove $f(n) \geq 1$, $NSPACE(f(n)) \subseteq SPACE(f^2(n))$.

Una mdt deterministica H può simulare una mdt non deterministica N usando soltanto spazio quadratico.

DIMOSTRAZIONE:

Il primo tentativo è quello di simulare tutti i possibili rami di computazione della Nmdt.

Occorre tenere traccia delle configurazioni non deterministiche della computazione. Ha una configurazione che usa spazio $f(n)$ può richiedere $2^{O(f(n))}$ passi ed ogni passo può essere non-deterministico. Le scelte fatte per percorrere tutti i possibili cammini sono esponenziali in numero, anche se lo spazio necessario per collocarli è polinomiale. Dunque, questo tentativo richiede spazio esponenziale.

Il secondo tentativo consiste nel verificare se una Nmdt N può passare da una configurazione c_1 ad una c_2 in un numero di passi minore o uguale ad un numero t , usando spazio $f(n)$. Questo è chiamato **problema della nesa** (reachability problem). Se c_1 è la configurazione iniziale e c_2 è configurazione finale, e t è il numero massimo di passi, allora possiamo stabilire se la macchina accetta il proprio input.

A tal fine forniamo un algoritmo ricorsivo deterministico che risolve il problema della nesa. Questo opera cercando una configurazione intermedia c_m , e verificando ricorsivamente se c_1 può portare a c_m in al più $t/2$ passi, e se c_m può portare a c_2 in al più $t/2$ passi. Questo algoritmo richiede spazio per memorizzare la chiamata ricorsiva. Il livello della ricorsione usa spazio $O(f(n))$ per memorizzare una configurazione. La profondità della ricorsione è $\log t$. Risulta $t = 2^{O(f(n))}$, quindi $\log t = O(f(n))$. Pertanto la simulazione deterministica usa spazio $O(f^2(n))$.

Definiamo la mdt deterministica H che simula la Nmdt N. La macchina H utilizza la procedura CANFIELD, che verifica se una delle due configurazioni di N può dare un'altura entro uno specifico numero di passi.

Sia w una stringa di input per N . Per la configurazione c_1 e c_2 di N , e l'intero t , $CANYIELD(c_1, c_2, t)$ dà in output accetta se N può passare da c_1 alla c_2 in t o meno passi, altrimenti un qualche cammino non deterministico. Altrimenti dà in output rifiuta. Per convenienza, assumiamo che t sia una potenza di 2.

$CANYIELD =$ "Su input c_1, c_2 e t :

1. Se $t=1$, allora verifica se $c_1=c_2$ o se c_1 dà c_2 in un passo. Accetta se uno dei due test ha successo, rifiuta se falliscono entrambi.
2. Se $t>1$, allora per ogni configurazione c_m di N usando spazio $\frac{t}{2}(n)$:
3. Esegui $CANYIELD(c_1, c_m, \frac{t}{2})$,
4. Esegui $CANYIELD(c_m, c_2, \frac{t}{2})$,
5. Se i passi 3 e 4 sono entrambi accettati, allora accetta.
6. Se non ha ancora accettato, rifiuta."

Per definire H , prima modifichiamo N in modo tale che, quando accetta, cancella il proprio mostro e muove la testina sulla cella più a sinistra, emittendo in output.

Dandogli come start la configurazione iniziale di N su w . Selezioniamo una costante d in modo tale che N abbia al più $2^{d(n)}$ configurazioni usando un mostro di $\frac{t}{2}(n)$ celle, dove n è la lunghezza di w .

$H =$ "Su input w :

1. Fornisce in output il risultato di $CANYIELD(\text{start}, \text{accept}, 2^{d(n)})$."

Ogni qualvolta $CANYIELD$ invoca se stessa ricorsivamente, memorizza il numero della fase corrente ed i valori di c_1, c_2 e t in una pila, in modo tale che questi valori possano essere recuperati dalla chiamata ricorsiva. Ciascun livello della ricorsione usa perciò uno spazio aggiuntivo pari a $O(\frac{t}{2}(n))$. Il numero delle chiamate ricorsive è invece pari a $\log_2 2^{d(n)} = d(n)$. Dunque, lo spazio totale usato da N : $d(n) \times O(\frac{t}{2}(n)) = O(d^2(n))$.

Una difficoltà tecnica sorge in questo argomento perché l'algoritmo H deve conoscere il valore di $\frac{t}{2}(n)$ quando invoca $CANYIELD$. Possiamo gestire questa difficoltà modificando H in modo tale che torni con $\frac{t}{2}(n)=1, 2, 3, \dots$. Per ciascun valore $\frac{t}{2}(n)=i$, usa $CANYIELD$ per stabilire se la configurazione di accettazione è raggiungibile. Inoltre, l'algoritmo usa $CANYIELD$ per stabilire se N usa almeno spazio $i+1$ verificando se N può raggiungere qualunque delle configurazioni di lunghezza $i+1$ dalla config. iniziale. Se la configurazione finale è raggiungibile, H accetta; se nessuna configurazione di lunghezza $i+1$ è raggiungibile, H rifiuta; e altrimenti, H continua con $\frac{t}{2}(n)=i+1$. $\square$

10. Dati due linguaggi A e B , lo *shuffle perfetto* di A e B è il linguaggio

$\{w \mid w = a_1 b_1 \dots a_k b_k, \text{ con } a_1 \dots a_k \in A \text{ e } b_1 \dots b_k \in B, \text{ per ogni } a_i, b_i \in \Sigma\}$; dimostrare che la classe dei linguaggi regolari è chiusa rispetto allo shuffle completo (4 punti).

Dati due linguaggi A e B , lo shuffle perfetto di A e B è il linguaggio $\{w \mid w = a_1 b_1 \dots a_k b_k, \text{ con } a_1 \dots a_k \in A \text{ e } b_1 \dots b_k \in B, \forall a_i, b_i \in \Sigma\}$; dimostrare che la classe dei linguaggi regolari è chiusa rispetto allo shuffle completo (4 punti).

Dati due linguaggi regolari, posso sempre costruire l'automa $(Q_{SH}, \Sigma_{SH}, \delta_{SH}, q_{0_{SH}}, F_{SH})$ per il linguaggio L_{SH} , dove:

- $Q_{SH} = (p, q), p \in Q_A, q \in Q_B$
- $\Sigma_{SH} = \Sigma_A \cup \Sigma_B$
- $q_{0_{SH}} = (p, q), p = q_{0_A}, q = q_{0_B}$
- $\delta((p, q), a) = (\delta_A(p, a), q)$
 $\delta((p, q), b) = (p, \delta_B(q, b))$
- $F_{SH} = \{(p, q), q \in F_B\}$

Il processo è simile a una costruzione per sottoinsiemi, dove quello che facciamo è andare a costruire un automa con stati "composti" in cui si alternano gli stati di A e B . Lo stato iniziale è lo stato iniziale di A , mentre lo stato finale è lo stato finale di B

11. Fornire una definizione operativa di macchina di Turing universale (4 punti).

Una mdT è codificata con una sequenza di 0 ed 1 e l'insieme delle stringhe binarie che descrivono una mdT forma un linguaggio.
 La mdT universale ha input la codifica di una mdT di tipo riconoscitore T ed esegue una computazione con esito uguale a quello di $T(X)$.

La macchina usa 4 nastri:

1. N_1 , su cui è memorizzata la descrizione di T (codifica)
2. N_2 , su cui è memorizzato l'input x di T
3. N_3 , su cui è memorizzato lo stato attuale di T
4. N_4 , dove è scritto lo stato di accettazione di T

Abbiamo 6 passi da svolgere:

1. Copio lo stato iniziale della macchina T e lo metto sullo stato attuale di T (N_3) e lo stato di accettazione su N_4
2. Se sullo stato q_1 c'è lo stesso simbolo su N_1 ed N_3 entro nello stato corretto. Inoltre se c'è lo stesso simbolo sui nastri N_1 ed N_2 allora è l'istruzione giusta e lo scriverò
3. Nello stato q_{scrivi} esegue il resto dell'istruzione
4. Nello stato q_{cambia} stato dobbiamo cambiare stato
5. Nello stato q_{muovi} dobbiamo spostarci sul nastro N_2
6. Nello stato $q_{riavvolgi}$ dobbiamo tornare all'inizio delle regole e ripetere

12. Il linguaggio $A_{CFG} = \{(G, w) \mid G \text{ è una}$

CFG che genera $w\}$ è decidibile? (4 punti).

$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ è una CFG che genera la stringa } w \}$

TEOREMA: A_{CFG} è un linguaggio decidibile.

DIMOSTRAZIONE:

Costruiamo una mdT che simula tutte le derivazioni di G e verifica che sia generata w .

Se w in $L(G)$, prima o poi la trova;

Se w non in $L(G)$, la macchina diverge.

la mdT S per A_{CFG} :

1. Converto G in una grammatica equivalente in forma normale di Chomsky;
2. Lista tutte le derivazioni di $2n-1$ passi, dove n è la lunghezza di w ; tranne se $n=0$, in tal caso lista tutte le derivazioni di un passo;
3. Se una di tali derivazioni genera w , accetta; altrimenti rifiuta. $\square$

13. Ogni linguaggio context free è decidibile. COME ESEMPIO FAI QUELLA DI SOPRA

DIMOSTRAZIONE. Sia G una CFG per A , progettiamo una TM M_G che decide A . Costruiamo una copia di G in M_G . Essa funziona come segue.

M_G = "Su input w :

1. Esegue la TM S su input $\langle G, w \rangle$.
2. Se questa macchina accetta, *accetta*; se essa rifiuta, *rifiuta*."

14. Ogni linguaggio CF è un elemento di P.

DIMOSTRAZIONE. Il seguente algoritmo D implementa l'idea della dimostrazione. Sia G una CFG in forma normale di Chomsky che genera il CFL L . Si supponga che S sia la variabile iniziale. (Ricordiamo che la stringa vuota viene gestita in modo speciale nella grammatica in forma normale di Chomsky. L'algoritmo gestisce il caso particolare in cui $w = \epsilon$ nella fase 1.) I commenti appaiono tra doppie parentesi quadre.

$D =$ "Su input $w = w_1 \dots w_n$:

1. Per $w = \epsilon$, se $S \rightarrow \epsilon$ è una regola, accetta; altrimenti, rifiuta.
[caso $w = \epsilon$]
2. Per $i = 1$ to n : [esamina ogni sottostringa di lunghezza 1]
3. Per ogni variabile A :
4. Testa se $A \rightarrow b$ è una regola, dove $b = w_i$.
5. Se sì, poni A in $table(i, i)$.
6. Per $l = 2$ to n : [l è la lunghezza della sottostringa]
7. Per $i = 1$ to $n - l + 1$: [i è la posizione iniziale della sottostringa]
8. Sia $j = i + l - 1$. [j è la posizione finale della sottostringa]
9. Per $k = i$ to $j - 1$: [k è la posizione di separazione]
10. Per ogni regola $A \rightarrow BC$:
11. Se $table(i, k)$ contiene B e $table(k + 1, j)$ contiene C , pone A in $table(i, j)$.
12. Se S è in $table(1, n)$, accetta; altrimenti, rifiuta."

15. Dimostrare che QBF è PSPACE completo (4 punti).

$TAQBF = \{ \langle \phi \rangle \mid \phi \text{ è una formula booleana chiusa vera} \}$

TEOREMA: TAQBF è PSPACE-completo.

DIMOSTRAZIONE:

$T =$ "Su input $\langle \phi \rangle$, una formula booleana completamente quantificata:

1. Se ϕ non ha quantificatori, valutare ed eventualmente accettare,
2. Se $\phi = \exists x. \psi$, chiama T su ψ con 0 (e poi 1) al posto di x , se uno dei due accetta, allora accetta; altrimenti rifiuta,
3. Se $\phi = \forall x. \psi$, chiama T su ψ con 0 (e poi 1) al posto di x , se tutti e due accettano, allora accetta; altrimenti rifiuta."

L'algoritmo T decide TAQBF. Lo spazio totale utilizzato è $O(m)$, dove m è il numero di variabili presenti in ϕ .

TAQBF è PSPACE-hard.

Sia A un linguaggio deciso da una mdT M in spazio n^k per qualche costante k . Costruisco un $tblQBF$ per la computazione di M su w , ed una formula ϕ che esprime un $tblQBF$ accettabile. Il $tblQBF$ è lungo $O(n^k)$, ma è profondo 2^n .

Se c_1 e c_2 sono due configurazioni di variabili, la formula $\phi_{c_1, c_2, k}$ è vera se e solo se M può andare da c_1 a c_2 in al più k passi.

Allora possiamo definire ϕ come la formula $\phi_{c_{start}, c_{accept}, n}$, dove $n = 2^{2^{2^n}}$ per una costante n , scelta in modo tale che M abbia non più di $2^{2^{2^n}}$ configurazioni possibili su

un input di lunghezza n .

La formula codifica i contenuti della cella di una configurazione come nella prova del teorema di Cook e Levin.

Se $k = 1$, costruisco $\phi_{c_1, c_2, k}$ ($c_1 = c_2$) oppure vedo da c_1 a c_2 in un passo.

Se $k > 1$, $\exists m \in \mathbb{N}$ tale che $k \leq m$.

Per ridurre la dimensione della formula, usiamo $\forall$ in aggiunta a $\exists$. Sia $\phi_{c_1, c_2, k} = \exists m \forall (c_3, c_4) \in \{c_1, m\} \times \{c_2, k\} [\phi_{c_3, c_4, k-1}]$.

Per calcolare la dimensione della formula $\phi_{c_{start}, c_{accept}, n}$ dove $n = 2^{2^{2^n}}$, notiamo che ciascun livello della ricorsione aggiunge una porzione della formula che è lineare nella dimensione delle configurazioni, ed è perciò di dimensione $O(n)$. Il numero di livelli della ricorsione è $\log_2(2^{2^{2^n}})$, cioè $O(2^n)$. Quindi la dimensione della formula risultante è $O(2^n \cdot n)$. $\square$

16. x è un prefisso proprio di y se $\exists z$ tale che $y = xz$ e $x \neq y$;

$\text{NOPREFIX}(A) = \{w\hat{1}A \mid \text{nessun prefisso proprio di } w \text{ appartiene ad } A\}$.

Dimostrare: se A è regolare, allora $\text{NOPREFIX}(A)$ è regolare. (4 punti)

1.45 (a) Sia $M = (Q, \Sigma, \delta, q_0, F)$ un DFA che riconosce A , dove A è un linguaggio regolare. Costruiamo $M' = (Q', \Sigma, \delta', q_0', F')$ che riconosce $\text{NOPREFIX}(A)$ come segue:

1. $Q' = Q$.
2. Per $r \in Q'$ e $a \in \Sigma$, definiamo $\delta'(r, a) = \begin{cases} \{\delta(r, a)\} & \text{se } r \notin F \\ \emptyset & \text{se } r \in F. \end{cases}$
3. $q_0' = q_0$.
4. $F' = F$.

17. Dimostrare che la classe dei linguaggi T-riconoscibili è chiusa rispetto all'unione. (4 punti)

3.15 Per ogni coppia di linguaggi decidibili L_1 e L_2 , siano M_1 e M_2 le TM che li decidono. Costruiamo una TM M' che decide l'unione di L_1 e L_2 :

“Su input w :

1. Esegue M_1 e M_2 alternativamente su passo dopo passo. Se una accetta, accetta. Se entrambe si fermano e rifiutano, rifiuta.”

Se M_1 accetta w oppure M_2 accetta w , allora M' accetta w perché la TM che accetta arriva al suo stato-accetta dopo un numero finito di passi. Si noti che se entrambe M_1 e M_2 rifiutano e una va in loop, allora anche M' andrà in loop.

18. Dimostrare l'esistenza di un linguaggio indecidibile (4 punti). **FORSE**

DEF: Un linguaggio è **coTuring-riconoscibile** se è il complemento di un linguaggio Turing-riconoscibile.

TEOREMA: Un linguaggio è decidibile se e solo se è Turing-riconoscibile e coTuring-riconoscibile.

DIMOSTRAZIONE:

Se A è decidibile, possiamo facilmente vedere che sia A che il suo complemento $\bar{A}$ sono Turing-riconoscibili.

Qualsiasi linguaggio decidibile è Turing-riconoscibile, e il complemento di un linguaggio decidibile è a sua volta decidibile.

Per la direzione inversa, se entrambi A e $\bar{A}$ sono Turing-riconoscibili, indichiamo con M_1 il riconoscitore per A e con M_2 il riconoscitore per $\bar{A}$. Per la macchina A definiamo un decisore H che, esegue sia M_1 che M_2 su input w in parallelo, e se M_1 accetta, accetta; se M_2 accetta, rifiuta.

Quando H alterna la simulazione di un passo di M_1 con un passo di M_2 e continua finché una delle due accetta. Poiché H si ferma ogni volta che M_1 accetta oppure M_2 accetta, allora H si ferma sempre quindi è un decisore. Inoltre, accetta tutte le stringhe in A e respinge quelle che non lo sono. Quindi H è un decisore per A , e pertanto A è decidibile.

19. Un triangolo in un grafo non orientato è una clique di dimensione 3. Dimostrare

che TRIANGLE è in P, con $\text{TRIANGLE} = \{ \langle G \rangle \mid G \text{ contiene un triangolo} \}$ (4 punti).

Per dimostrare che TRIANGLE è in P, possiamo usare questo algoritmo:

Algoritmo su input $\langle G \rangle$ per ogni tripla di nodi u, v, w distinti controlla se esistono gli archi uv - vw - uw . Appena trova un triangolo accetta, se ha finito le terne e non ha accettato rifiuta.

Ci saranno quindi $n(n-1)(n-2)/3!$ terne dove n è il numero di nodi di G .

Quindi la complessità sarà $O(n^3)$ quindi polinomiale

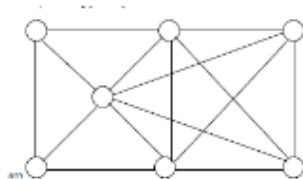
15. Un triangolo in un grafo non orientato è una clique di dimensione 3. Dimostrare che TRIANGLE è in P, con $\text{TRIANGLE} = \{\langle G \rangle \mid G \text{ contiene un triangolo}\}$ (4 punti).

IN GENERALE

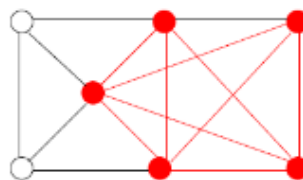
Una clique in un grafo non orientato è un sottografo in cui ognicoppia di nodi è collegata da un arco

Una k clique è una clique che contiene k nodi

$\text{CLIQUE} = \{(G, k) \mid G \text{ contiene una } k \text{ clique}\}$



c'è una 5 clique?



Trovare tutte le permutazioni di 5 elementi e verificare che ogni elemento è connesso con tutti gli altri... $(5! \times (5-1) \times (4-1) \dots \times 1)$ circa exp.)

L'algoritmo controlla ogni nodo del grafo e, per ogni coppia di nodi adiacenti, controlla se esiste un arco diretto tra di loro. Se viene trovato un triangolo, l'algoritmo termina e restituisce "true". Se dopo aver esaminato tutti i nodi non viene trovato alcun triangolo, l'algoritmo termina e restituisce "false".

RISOLUZIONE DEL PROBLEMA RICHIESTO

Per dimostrare che il problema **TRIANGLE** è in P, ovvero che può essere risolto in tempo polinomiale, possiamo utilizzare l'algoritmo seguente:

Per ogni nodo v nel grafo G , esegui i seguenti passaggi:

1. Per ogni coppia di nodi adiacenti u e w a v , controlla se esiste un arco diretto tra u e w .
 - Se esiste, abbiamo trovato un triangolo nel grafo e possiamo restituire "true".
2. Se non è stato trovato un triangolo, passa al prossimo nodo.
 - Se nessun triangolo è stato trovato dopo aver esaminato tutti i nodi, restituisci "false".

L'algoritmo è polinomiale perché richiede un numero di operazioni che dipende linearmente dal numero di nodi e archi nel grafo.

Quindi, l'algoritmo risolve il problema **TRIANGLE** in tempo polinomiale e dimostra che **TRIANGLE** è in P.

20. $\text{ETM} = \{ \langle M \rangle \mid M \text{ è una MdT e } L(M) = \emptyset \}$ è decidibile? (4 punti)

DIMOSTRAZIONE Descriviamo la macchina modificata già descritta nell'idea di dimostrazione usando la nostra notazione standard. Chiamiamola M_1 .

M_1 = "Su input x :

1. Se $x \neq w$, *rifiuta*.
2. Se $x = w$, esegue M su input w e *accetta* se M accetta."

Questa macchina ha la stringa w come parte della sua descrizione. Essa verifica se $x = w$ nel modo ovvio, attraverso la scansione dell'input e confrontandolo carattere per carattere con w per determinare se coincidono. Mettendo insieme tutto questo, assumiamo che la TM R decide E_{TM} e costruiamo la TM S che decide A_{TM} come segue.

S = "Su input $\langle M, w \rangle$, una codifica di una TM M e una stringa w :

1. Usa la descrizione di M e w per costruire la TM M_1 descritta sopra.
2. Esegue R su input $\langle M_1 \rangle$.
3. Se R accetta, *rifiuta*; se R rifiuta, *accetta*."

Notate che S deve essere effettivamente in grado di calcolare una descrizione di M_1 da una descrizione di M e w . Può farlo, perché ha bisogno solo di aggiungere ad M alcuni stati in più che svolgono il test $x = w$. Se R

fosse un decisore per E_{TM} , S sarebbe un decisore per A_{TM} . Un decisore per A_{TM} non può esistere, quindi sappiamo che E_{TM} deve essere indecidibile.

21. Dato il linguaggio $E_{TM} = \{ \langle M \rangle \mid M \text{ è una mdT e } L(M) = \emptyset \}$, dimostrare che il suo complemento è Turing-riconoscibile

4.5 Sia $s_1, s_2, \dots$ la lista di tutte le stringhe in Σ^* . La seguente TM riconosce $\overline{E_{TM}}$.

"Su input $\langle M \rangle$, dove M è una TM:

1. Ripete per $i = 1, 2, 3, \dots$
2. Esegue M per i passi su ogni input, $s_1, s_2, \dots, s_i$.
3. Se M ne ha accettato almeno uno, *accetta*. Altrimenti, *continua*."

In sintesi, il complemento del linguaggio $\{ \langle M \rangle \mid M \text{ è una macchina di Turing e } L(M) = \emptyset \}$ è Turing-riconoscibile, poiché esiste un algoritmo che riconosce il complemento utilizzando una macchina di Turing non deterministica.

22. Dimostrare che EQDFA è decidibile

$E_{DFA} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ sono DFA e } L(A) = L(B) \}$

TEOREMA: E_{DFA} è un linguaggio decidibile.

DIMOSTRAZIONE:

Costruiamo un DFA C a partire da A e B , dove C accetta solo le stringhe accettate da A o da B , ma non da entrambi.

Il linguaggio di C è $L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$, cioè la differenza simmetrica di $L(A)$ e $L(B)$.

Quindi $L(C) = \emptyset$ se e solo se $L(A) = L(B)$.

Costruiamo il DFA C mediante la costruzione fatta per dimostrare che la classe dei linguaggi regolari risulta chiusa rispetto alle operazioni di complemento, unione ed intersezione.

Per verificare se $L(C)$ è vuoto o meno possiamo usare il teorema precedente. Quindi:

1. Costruisce il DFA C ,
2. Esegue la MdT M su input $\langle C \rangle$,
3. Se M accetta, accetta; altrimenti rifiuta. $\square$

23. $E_{CFG} = \{ G \mid \text{il linguaggio generato dalla CFG } G \text{ è vuoto} \}$ è decidibile

$E_{CFG} = \{ G \mid G \text{ è una CFG ed } L(G) = \emptyset \}$

TEOREMA: E_{CFG} è un linguaggio decidibile.

DIMOSTRAZIONE:

Costruiamo una MdT R che:

1. Marca tutti i simboli terminali in G ;
2. Ripete fin quando nessuna nuova variabile viene marcata:
- marca una qualsiasi variabile A tale che G ha una regola $A \rightarrow U_1 U_2 \dots U_k$ ed ogni simbolo $U_1, \dots, U_k$ è già stato marcato;
3. Se la variabile iniziale non è segnata, accetta; altrimenti rifiuta. $\square$

24. $REGULARTM = \{ M \mid \text{tm e } L(M) \text{ è regolare} \}$ è indecidibile

DIMOSTRAZIONE Definiamo R come una TM che decide $REGULAR_{TM}$ e costruiamo una TM S che decide A_{TM} . Allora S funziona come segue.

232 CAPITOLO 5 -- Riducibilità

$S =$ "Su input $\langle M, w \rangle$, dove M è una TM e w è una stringa:

1. Costruisce la seguente TM M_2 .

$M_2 =$ "Su input x :

1. Se x ha la forma $0^n 1^n$, accetta.
2. Se x non ha tale forma, esegue M su input w e accetta se M accetta w ."

2. Esegue R su input $\langle M_2 \rangle$.

3. Se R accetta, accetta; se R rifiuta, rifiuta."

25. $HALT_{TM} = \{ \langle M, w \rangle \mid \text{la mdT } M \text{ si ferma su } w \}$ indecidibile.

$HALT_{TM} = \{ \langle M, w \rangle \mid \text{la mdT } M \text{ si ferma su } w \}$

TEOREMA: $HALT_{TM}$ è un linguaggio indecidibile.

DIMOSTRAZIONE:

Supponiamo per assurdo che $HALT_{TM}$ sia decidibile. Allora esiste un decisore H per $HALT_{TM}$.

Dunque, assumiamo che H sia una mdT, dove $H(\langle M, w \rangle) = \begin{cases} \text{accetta} & \text{se } H \text{ accetta } w \\ \text{rifiuta} & \text{se } H \text{ non accetta } w \end{cases}$

Definiamo la macchina H' che usa H : essa accetta se H rifiuta e rifiuta se H accetta.

TEOREMA: $\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ è una mdT e } M \text{ si ferma su input } w \}$ è indecidibile.

DIMOSTRAZIONE:

Supponiamo per assurdo che HALT_{TM} sia decidibile. Allora esiste una decisione R per HALT_{TM} .

Usiamo R per costruire S , una mdT che decide A_{TM} . Questa mdT:

1. esegue la mdT R su input $\langle M, w \rangle$,
2. se R rifiuta, rifiuta,
3. se R accetta, simula M su w finché non si ferma,
4. se M ha accettato, accetta, se M ha rifiutato, rifiuta.

Se R decide HALT_{TM} , allora S decide A_{TM} . Poiché A_{TM} è indecidibile, allora lo è anche HALT_{TM} .

Questa meglio

26. $A = \{ \langle M \rangle \mid M \text{ è un DFA e non accetta nessuna stringa che contiene un numero dispari di 1} \}$; A è decidibile?

4.30 La seguente TM decide A .

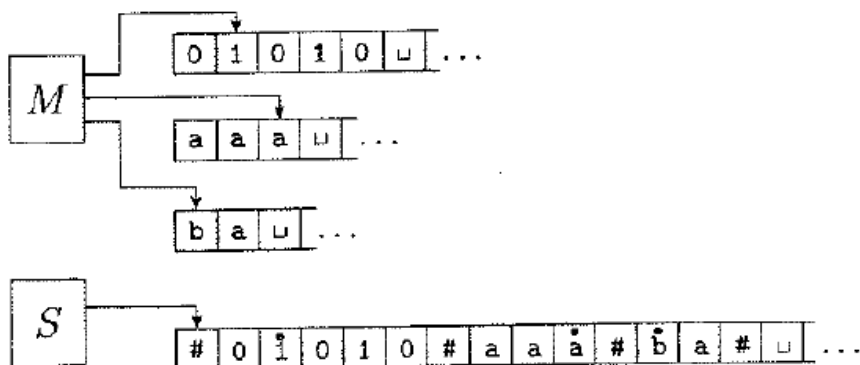
“Su input $\langle M \rangle$:

1. Costruisce il DFA O che accetta ogni stringa contenente un numero dispari di 1.
2. Costruisce un DFA B tale che $L(B) = L(M) \cap L(O)$.
3. Testa se $L(B) = \emptyset$ usando il decisore T per E_{DFA} del Teorema 4.4.
4. Se T accetta, accetta; se T rifiuta, rifiuta.”

MDT multinastro = MDT singolo nastro

Supponiamo di avere una MDT M con k nastri, e vogliamo convertirla in una MDT S a singolo nastro.

Allora S dovrà simulare l'effetto di k nastri su un singolo nastro. La macchina S utilizzerà il nuovo simbolo $\#$ come delimitatore per separare i contenuti dei diversi nastri. Oltre al contenuto dei nastri, però, la macchina S deve tenere traccia delle posizioni delle varie testine. Per fare ciò, scrive un simbolo con un punto sopra per contrassegnare la posizione in cui si troverebbe la testina in quel momento.



27. REALPRIME APPARTIENE A p

REALPRIME = $\{ \langle x, y \rangle \mid x \text{ e } y \text{ sono relativamente primi} \}$

TEOREMA: REALPRIME $\in P$

DIMOSTRAZIONE:

Algoritmo euclideo $E =$ "Su input $\langle x, y \rangle$, dove x e y sono numeri naturali in binario:

1. Ripete finché $y \neq 0$:

2. Pon $x \leftarrow x \bmod y$,

3. Scambia x e y ,

4. Output x ."

L'algoritmo R risolve REALPRIME, utilizzando E come sottoprocedura.

$R =$ "Su input $\langle x, y \rangle$, dove x e y sono numeri naturali in binario:

1. Esegue E su $\langle x, y \rangle$;

2. Se il risultato è 1, accetta. Altrimenti, rifiuta."

Per analizzare la complessità di tempo di E , mostriamo che nello step 2 ogni esecuzione dimezza x e poi risulta $x < y$. Dopo lo step 3, risulta $x > y$ perché i due valori sono stati scambiati. Quindi, le moltiplicazioni di volte in cui lo step 2 e 3 sono eseguite risulta pari al minimo tra $\log_2 x$ e $\log_2 y$. Quindi $O(\min)$. $\square$

28. SQUARE APPARTIENEN IN P **FARE**

29. $PATH = \{(G,s,t) \mid G \text{ è un grafo orientato che contiene un cammino diretto da } s \text{ a } t\}$; in quale classe di complessità si colloca $PATH$? (4 punti)

$PATH$ è in P

TEOREMA: $PATH \in P$

Dimostrazione:

Algoritmo $H =$ "Su input (G,s,t) , dove G è un grafo orientato con nodi s e t :"

1. Marca il nodo s ;
2. Ripete il seguente passo fino a quando nessun nuovo nodo viene marcato.
3. Scansiamo tutto gli archi di G . Se trova un arco (a,b) che va da un nodo a marcato ad un nodo b non marcato, marca il nodo b ;
4. Se t è marcato, accetta. Altrimenti, rifiuta."

Le fasi 1 e 4 sono eseguite una sola volta. la fase 3 è eseguita al più m volte, dove m è il numero dei nodi. $\square$

30. $HAMPATH$ è in NP . **FATTO**

TEOREMA: $HAMPATH \in NP$.

Dimostrazione:

$N =$ "Su input (G,s,t) , dove G è un grafo orientato con nodi s e t :"

1. Scegliere m numeri $p_1, \dots, p_m$ tra 1 a m ;
2. Se ci sono ripetizioni, rifiuta;
3. Se una tra $s=p_1$ e $t=p_m$ è falsa, rifiuta;
4. Per ogni i , se (p_i, p_{i+1}) non è un arco, rifiuta, altrimenti accetta." $\square$

Ogni passaggio è risolto in tempo polinomiale $O(n)$, quindi il problema è risolto in tempo polinomiale non deterministico.

31. Fornire un esempio di problema in NP (4 punti). **SCRIVI CLIQUE**

32. CLIQUE è in NP.

TEOREMA 7.24

CLIQUE è in NP.

IDEA. La clique è il certificato.

DIMOSTRAZIONE. Il seguente algoritmo è un verificatore V per *CLIQUE*.

$V =$ "Su input $\langle G, k \rangle, c$:

1. Controlla se c è un sottografo con k nodi in G .
2. Controlla se G contiene tutti gli archi tra i nodi in c .
3. Se entrambe le condizioni sono verificate, *accetta*; altrimenti, *rifiuta*."

DIMOSTRAZIONE ALTERNATIVA. Se preferite pensare a NP in termini di macchine di Turing di tempo polinomiale non deterministico, si può dimostrare questo teorema dandone una che decide *CLIQUE*. Osservate la somiglianza tra le due prove.

$N =$ "Su input $\langle G, k \rangle$, dove G è un grafo:

1. Non deterministicamente seleziona un sottoinsieme c di k nodi di G .
2. Controlla se G contiene tutti gli archi tra i nodi in c .
3. Se sì, *accetta*; altrimenti, *rifiuta*."

Ogni passaggio è risolto al più in tempo polinomiale $O(n)$.

33. $SUBSET-SUM = \{(S,t) \mid S=\{x_1, \dots, x_k\} \text{ ed esiste } \{y_1, \dots, y_m\} \subseteq S \text{ con } \sum y_i = t\}$; in quale classe di complessità si colloca $SUBSET-SUM$? (4 punti)

$SUBSET-SUM$ è in NP. **FATTO**

SUBSET-SUM è in NP.

IDEA. Il sottoinsieme è il certificato.

DIMOSTRAZIONE. L'algoritmo seguente è un verificatore V per *SUBSET-SUM*.

$V =$ "Su input $\langle S, t, c \rangle$:

1. Verifica se c è una collezione di numeri la cui somma è t .
2. Verifica se S contiene tutti i numeri in c .
3. Se le condizioni sono entrambe vere, *accetta*; altrimenti, *rifiuta*."

DIMOSTRAZIONE ALTERNATIVA. Possiamo anche dimostrare questo teorema esibendo una macchina di Turing di tempo polinomiale non deterministico per *SUBSET-SUM* come segue.

$N =$ "Su input $\langle S, t \rangle$:

1. In modo non deterministico seleziona un sottoinsieme c dei numeri in S .
 2. Verifica se c è una collezione di numeri la cui somma è t .
 3. Se la condizione è vera, *accetta*; altrimenti, *rifiuta*."
-

Ogni passaggio è risolto al più in tempo polinomiale $O(n)$.

34. Un linguaggio è in NP se e solo se viene deciso in tempo polinomiale da una mdt non deterministica. **FATTO** (MEGLIO LIBRO)

Un linguaggio è in NP sse esso viene deciso in tempo polinomiale da una macchina di Turing non deterministica.

IDEA. Mostriamo come convertire un verificatore di tempo polinomiale in una NTM equivalente e viceversa. La NTM simula il verificatore per indovinare il certificato. Il verificatore simula la NTM utilizzando il ramo di computazione accettante come certificato.

DIMOSTRAZIONE. Per la parte diretta di questo teorema, assumiamo $A \in \text{NP}$ e mostriamo che A è deciso da una NTM N avente tempo polinomiale. Sia V il verificatore polinomiale per A la cui esistenza è assicurata dalla definizione di NP. Si supponga che V è una TM avente tempo di esecuzione n^k e costruiamo N come segue.

$N =$ "Su input w di lunghezza n :

1. In maniera non deterministica seleziona una stringa c di lunghezza al più n^k .
2. Esegue V su input $\langle w, c \rangle$.
3. Se V accetta, *accetta*; altrimenti, *rifiuta*."

Per dimostrare la direzione inversa del teorema, supponiamo che A è deciso da una NTM N avente tempo polinomiale e costruiamo come segue un verificatore V avente tempo polinomiale.

$V =$ "Su input $\langle w, c \rangle$, dove w e c sono stringhe:

1. Simula N su input w , trattando ogni simbolo di c come la descrizione di una scelta non deterministica da formulare ad ogni passo (come nella dimostrazione del Teorema 3.16).
2. Se questo ramo di computazione di N accetta, *accetta*; altrimenti, *rifiuta*."

35. Se A minore pB e B appartiene P, allora A appartiene P

TEOREMA: Se $A \leq_p B$ e $B \in P$, allora $A \in P$.

DIMOSTRAZIONE:

Sia H l'algoritmo di tempo polinomiale che decide B ed f la riduzione di tempo polinomiale da A a B .

$N =$ "Su input w :

1. Calcola $f(w)$,
2. Esegui H con input $f(w)$ e dai in output qualsiasi cosa H dà in output."

Resulta $w \in A$ ogni volta che $f(w) \in B$ perché f è una riduzione da A a B . Pertanto, H accetta $f(w)$ ogni volta che $w \in A$. $\square$

DEF: Un **litterale** è una variabile booleana o una variabile booleana negata, come x o $\bar{x}$.

Una **clausola** consiste in diverse litterali connesse tramite operatori $\vee$, come $(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee x_4)$.

Una **formula normale congiuntiva**, ed è detta una **formula cnf**, se comprende diverse clausole connesse tramite operatori $\wedge$.

36. 3SAT è riducibile in tempo polinomiale a clique

3SAT = $\{ \phi \mid \phi \text{ è una formula 3cnf soddisfacibile} \}$.

Made with Goodnotes

TEOREMA: 3SAT è riducibile in tempo polinomiale a clique.

DIMOSTRAZIONE:

Dobbiamo convertire formula in 3SAT in un grafo in cui c'è una clique che "corrisponde" ad assegnamento che soddisfa la formula.

Dato una formula: $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_1 \vee b_2 \vee c_2) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$

Il grafo è definito nel seguente modo:

- k gruppi di triple (tre modi),
- ogni modo in una tripla è un litterale della clausola associata,
- ogni modo ha etichetta 1/Litterale nella formula.

Ma anche mettiamo tutti i modi insieme:

- i modi della stessa tripla,
- i modi con etichette corrispondenti.

La formula è soddisfacibile se il grafo ha una k -clique.

Se ϕ ha un assegnamento che la soddisfa, allora $\exists$ litterale vero $\forall$ clausola. Prendo i modi corrispondenti che formano una k -clique.

Se il grafo ha una k -clique, allora nessuna coppia di modi della clique è nella stessa tripla ed ogni tripla contiene esattamente un modo della clique. Assegno T/F alla variabile in ϕ t.c. ogni litterale della clique è vero. $\Rightarrow$ l'assegnamento soddisfa ϕ . $\square$

37. Pumping lemma regolari.

TEOREMA PUMPING LEMMA:

Sia L un linguaggio regolare infinito. Esiste un intero m tale che, per ogni stringa w in L , con $|w| \geq m$, esistono x, y, z tali che $w = xyz$ con:

- $|xy| \leq m$, $|y| \geq 1$

- xy^iz in L

DIMOSTRAZIONE:

Sia $M = (Q, \Sigma, \delta, q_0, F)$ un DFA che riconosce A e sia m il numero di stati di M .

Sia $w = w_1w_2 \dots w_n$, una stringa in A di lunghezza n , dove $n \geq m$.

Sia $s_1 \dots s_{n+1}$ la sequenza di stati attraversata da M mentre elabora w , quindi $s_{i+1} = \delta(s_i, w_i)$, per $1 \leq i \leq n$. Questa sequenza ha lunghezza $n+1$, che è almeno $m+1$. Due tra i primi $m+1$

elementi della sequenza devono essere lo stesso stato, per il principio della piccolanza. Chiamiamo il primo di questi s_j ed il secondo w_l . Poiché s_j si presenta tra le prime

$m+1$ posizioni in una sequenza che inizia iniziando in s_1 , abbiamo $j \leq m+1$. Ora sia $x = w_1 \dots w_{j-1}$, $y = w_j \dots w_{l-1}$ e $z = w_l \dots w_n$.

Poiché x porta M da s_1 a s_j , y porta M da s_j a s_j e z porta M da s_j a s_{n+1} , che è uno stato accettabile, M deve accettare xy^iz per $i \geq 0$. Sappiamo che $j \geq 1$, perciò $|y| > 0$, e $j \leq m+1$, perciò $|xy| \leq m$. $\square$

Se A è un linguaggio regolare, allora esiste un numero p tale che se s è una qualsiasi stringa in A , con $|s| \geq p$, allora s si può scrivere come $s = uvw$, soddisfacendo le seguenti proprietà:

1- Per ogni $i \geq 0$, $u v^i w \in A$

2- $|v| > 0$

3- $|uv| \leq p$

Pumping lemma per i context free.

Capitolo 2 - Pumping lemma per i CFL

Un linguaggio L soddisfa le condizioni del pumping lemma per i linguaggi context-free se esiste $m > 0$ tale che per ogni stringa x in L di lunghezza almeno m esistono le stringhe u, v, w, y, z tali che:

$$x = uvwyz$$

$$|uv| \leq m$$

$$|v| \geq 1$$

$$\forall i \geq 0, uv^i w y^i z \in L$$

Dimostrazione:

Possiamo trovare due pezzi di una stringa sufficientemente lunga che iterate insieme producano stringhe ancora nel linguaggio.

Consideriamo S abbastanza lunga. L'albero deve essere alto perché un albero basso non può generare una stringa molto lunga.

Se l'albero è alto, allora c'è un percorso di molti molti rami, ma G ha un numero finito di variabili, quindi almeno una si deve ripetere lungo il percorso (R).

b è la massima lunghezza della parte destra della raga, oè il massimo branching possibile nel parse tree.

h è l'altezza del parse tree per la stringa s .

Un albero alto h con max branching b ha al massimo b^h foglie (quindi $|s| \leq b^h$).

Per avere una ripetizione di variabili deve essere $h > |V|$, e per ottenere questo prendiamo $|s| > b^{|V|}$.

Fissiamo $m = b^{|V|} + 1$. Se $|s| \geq m > b^{|V|}$ allora $h > |V|$. $\square$

Se A è un CFL, allora esiste un numero p tale che, se s è una qualsiasi stringa in A di lunghezza almeno p , allora s si può scrivere come $s = uvwxy$, e questa stringa soddisfa le seguenti proprietà:

- 1- Per ogni $i \geq 0$, $u v^i w x^i y \in A$
- 2- $|vx| > 0$
- 3- $|vwx| \leq p$

38. $A = \{ \langle M \rangle \mid M \text{ è un DFA e non accetta nessuna stringa che contiene un numero dispari di } 1 \}$; A è decidibile? (4 punti)

39. Se L e M sono linguaggi arbitrari, dimostrare che $L(M \cap N) = L M \cap L N$; usando questa proprietà distributiva, provare che $0 + 01^* = 01^*$ (4 punti).

40. Convertire in PDA la grammatica delle espressioni riportata in seguito (4 punti).

$$A \rightarrow a \mid b \mid Aa \mid Ab \mid A0 \mid A1$$

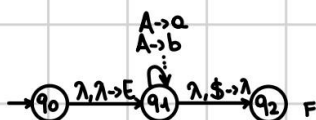
$$E \in A \mid E^*E \mid E+E \mid (E)$$

Se il linguaggio è context free, allora esiste un PDA che lo riconosce slide vedere

2) Convertire in PDA la grammatica

$$A \rightarrow a \mid b \mid Aa \mid Ab \mid A \mid A^*$$

$$E \rightarrow A \mid E \cdot E \mid E + E \mid (E)$$



41. Dimostrare che MdT multinastro e MdT a singolo nastro accettano la stessa classe di linguaggi (4 punti). **QUADERNO TUTTO TURING**

TEOREMA: Dato una macchina di Turing $M^{(k)}$ con k nastri, allora esiste una macchina di Turing M a singolo nastro che la simula.

DIMOSTRAZIONE:

Sia $M^{(k)}$ la MT multi-nastro così definita: $M^{(k)} = (\Sigma, \Gamma, k, q_0, F, \delta)$ ove supponiamo per ogni nastro i , $1 \leq i \leq k$ che l'alfabeto usato sia Σ_i .

Costruiamo una MT singolo nastro, avente $2k$ tracce: $M = (\Sigma', \Gamma', k', q_0', F', \delta')$ con $\Sigma' = \{ \langle i, a \rangle \mid i \in \{1, \dots, k\}, a \in \Sigma_i \}$ e $\Gamma' = \{ \langle i, b \rangle \mid i \in \{1, \dots, k\}, b \in \Gamma_i \}$, cioè l'alfabeto è composto di k coppie di simboli (i, a) di cui $i \in \{1, \dots, k\}$ e $a \in \Sigma_i$ per ogni $1 \leq i \leq k$.

Il nastro di M' risulterà allora composto nel seguente modo:

$\forall i, 1 \leq i \leq k$, la traccia pari di indice $2i$ contiene la stringa presente sul nastro di indice i ;

$\forall i, 1 \leq i \leq k$, la traccia dispari di indice $2i-1$ contiene una stringa composta del solo simbolo $\downarrow$ rappresentante la posizione della testina del nastro di indice i .

Per simulare la funzione di transizione di $M^{(k)}$ della MT multi-nastro M' , la funzione di transizione δ' deve scrivere $2k$ simboli, uno per traccia. Quindi, in particolare, per simulare una transizione del tipo: $\delta^{(k)}(q_i, a_1, \dots, a_{2k}) = (q_j, b_1, \dots, b_{2k}, d_1, \dots, d_k)$ deve eseguire i seguenti passi:

1. rintracciare la posizione dei k simboli $\downarrow$ rappresentanti la posizione delle k testine della macchina $M^{(k)}$ nello stato q_i ;

2. scrivere i k simboli puntati dalle testine dei k nastri;

3. posizionare i k simboli $\downarrow$ nella posizione delle testine della macchina $M^{(k)}$ nello stato q_j ;

4. traslare di stato.

Quindi ad ogni passo di $M^{(k)}$, la macchina M' deve eseguire un numero di passi proporzionale alla distanza, in termini di numero di celle, tra i due simboli $\downarrow$ più lontani.

Ad ogni passo i simboli $\downarrow$ possono al più allontanarsi di 2 celle, quindi dopo t passi, nel caso pessimo, si saranno allontanati di $2t$ celle.

Quindi se la macchina multi-nastro $M^{(k)}$ compie t passi, il numero di passi eseguiti dalla macchina singolo-nastro M' per simulare sarà:

$$\sum_{i=1}^t 2i = 2 \sum_{i=1}^t i = 2 \cdot \frac{t(t+1)}{2} = t^2 + t = O(t^2).$$

La dimensione dell'alfabeto dei nastri dispari è 2, mentre quella dei nastri pari, per ogni nastro i è $|\Sigma_i|$.

Quindi, la dimensione dell'alfabeto Σ' usato da M' sarà pari al prodotto di tutte le cardinalità degli alfabeti dei singoli nastri: $|\Sigma'| = \prod_{i=1}^k |\Sigma_i| = O((\max |\Sigma_i|)^k)$.

Possiamo affermare che una MT multi-nastro può essere simulata da una MT singolo-nastro multi-traccia in un tempo quadratico usando un alfabeto di cardinalità esponenziale nel numero dei nastri.

42. Dato il linguaggio $\{ ww \mid w \in \{0,1\}^* \}$, dimostrare che non è regolare (4 punti).

Esempio: $L = \{ww \mid w \in \{0,1\}^*\}$

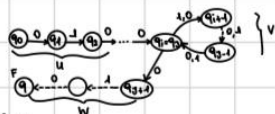
WW NON REGOLARE

Supponiamo per assurdo che L sia regolare. Allora $\exists M = (S, Q, \delta, q_0, F)$ t.c. $L = L(M)$. Sia $z = a^n b^n$ dove $|z| = n$, allora esiste un partizionamento $z = uvw$ t.c. $uv^i w \in L$ se vogliamo.

1) $|uv| \leq n$

2) $v \neq \epsilon$

Sia M definito come segue

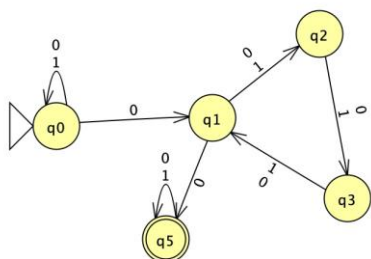


Da M noto: $u = w_1^i, v = w_1^{j-i}, w = w_1^{n-j} w_2^n$

Considero $uv^i w = w_1^i (w_1^{j-i})^i w_1^{n-j} w_2^n = w_1^i w_1^{i(j-i)} w_1^{n-j} w_2^n = w_1^{i+j-i^2} w_1^{n-j} w_2^n = w_1^{i+j-i^2+n-j} w_2^n = w_1^{n-i^2} w_2^n$ dove provare che $i+j-i^2 = n$. Quindi $uv^i w \in L$.

Dimostro che L non è regolare.

43. Definire l'automa a stati finiti che accetta le parole in $\{0, 1\}^*$ nelle quali figurano due zeri separati da un numero di posizioni multiplo di 3 (4 punti).



44. Dato A un linguaggio regolare, dimostrare se $\text{DOUBLE}(A) = \{ww \mid w \in A\}$ è regolare o meno (4 punti).

45. $\text{EDFA} = \{ |A| \mid A \text{ è un DFA e } L(A) = \emptyset \}$ è decidibile? (4 punti). **QUADERNO 45**

$\text{EDFA} = \{ |A| \mid A \text{ è un DFA e il linguaggio è vuoto} \}$

TEOREMA: EDFA è un linguaggio decidibile.

DIMOSTRAZIONE:

Costruiamo una MOT M che sulla codifica di A :

- manca lo stato iniziale di A ;
- ripete fin quando non vengono più mancata nuovi stati;
- manca qualsiasi stato che ha una transizione proveniente da uno stato già mancato;
- se nessuno stato di accettazione risulta mancato, accetta; altrimenti rifiuta. $\square$

46. Dimostrare che un linguaggio è context-free se e solo se esiste un automa a pila che lo riconosce **QUADERNO 32-33-34**

TEOREMA: Un linguaggio è context-free se e solo se esiste un automa a pila che lo riconosce

↳ due dimostrazioni

LEMMA 1: Se un linguaggio è context free, allora esiste un automa a pila che lo riconosce

Sia A un CFL $\Rightarrow$ esiste una grammatica G che genera A .

Dobbiamo costruire il PDA P che accetta in input w se w è generata da G

↳ ogni passo di una derivazione produce una stringa intermedia di variabili e terminali.

Descrizione informale di P :

1. Inserisce il simbolo marcatore $\$$ e la variabile iniziale sulla pila;

2. Ripetere:

a. se $\text{top} = V$, scegliere una regola per V e sostituire il dato destro nello stack;

b. se $\text{top} = a$ e $a =$ simbolo contenente in input, cancellarla; se $\text{top} = a$ e $a \neq$ simbolo contenente in input, fallire;

c. se $\text{top} = \$$, passa allo stato accettante.

Dimostrazione:

$P = (Q, \Sigma, \Gamma, q_{\text{start}}, F)$

Siamo q ed r stati del PDA e siamo a in Σ ed s in Γ . Vogliamo che il PDA vada da q a r quando legge a ed elimina s . Inoltre, deve inserire la stringa $u = u_1 \dots u_n$ sulla pila.

La mossa $(r, u) \in \delta(q, a, s)$ per dimostrare che quando q è lo stato in cui si trova l'automa, a è il prossimo simbolo di input ed s è il simbolo sulla cima della pila, il PDA può leggere a ed eliminare s , poi inserire la stringa u nella pila e passare nello stato r .

Gli stati di P sono $Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{accept}}\} \cup E$, dove E è l'insieme degli stati necessari per realizzare $(r, u) \in \delta(q, a, s)$.

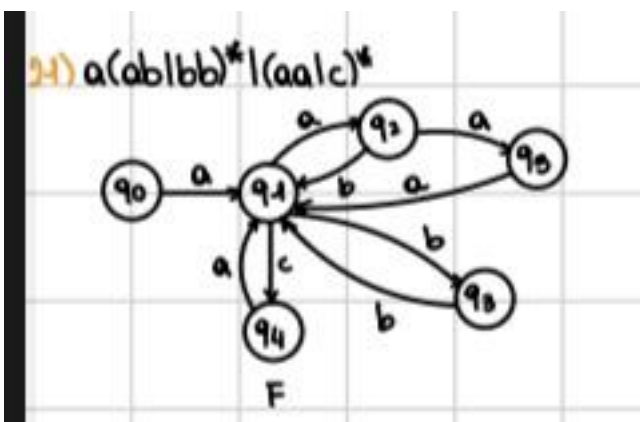
Per definire la funzione di transizione inizieremo la pila inserendo $\$$ e S : $\delta(q_{\text{start}}, \epsilon, \epsilon) = \{(q_{\text{loop}}, \$S)\}$.

Il primo bugger tratteremo il caso (a): poniamo $\delta(q_{\text{loop}}, \epsilon, A) = \{(q_{\text{loop}}, w)\}$ dove $A \rightarrow w$ è una regola in R .

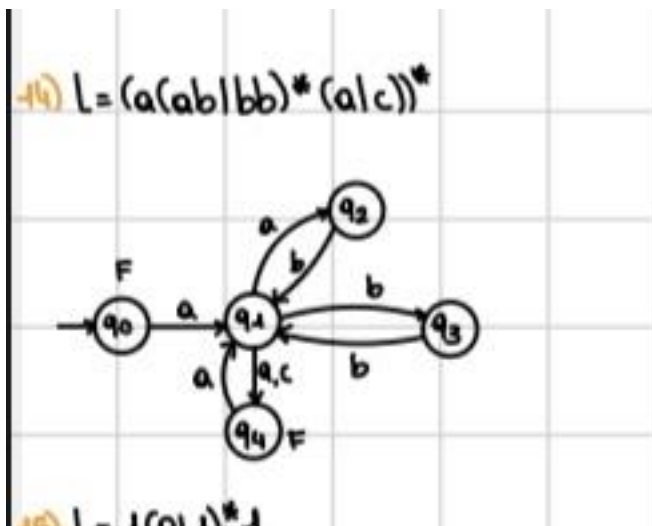
Il secondo bugger tratteremo il caso (b): poniamo $\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \epsilon)\}$.

Nel caso (c): poniamo $\delta(q_{\text{loop}}, \epsilon, \$) = \{(q_{\text{accept}}, \epsilon)\}$. $\square$

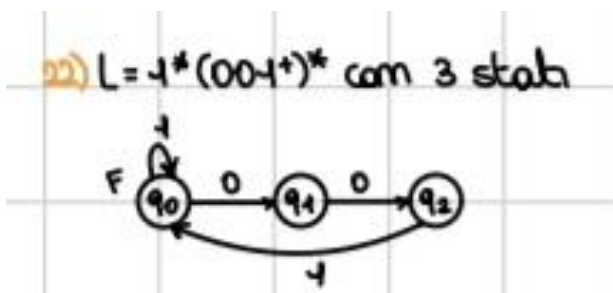
47. Scrivere il DFA che accetta stringhe nel linguaggio $(a(ab|bb)^*(a|c))^*$ (4 punti). **FATTA**



48. Scrivere il DFA che accetta stringhe nel linguaggio $(a(ab|bb)^*(aa|c))^*$ (4 punti). **FATTA**



49. Definire l'NFA per il linguaggio $1^*(001^+)^*$ con tre stati (4 punti).



50. Scrivere la definizione di DFA, di NFA, e definire il DFA che riconosce il linguaggio $(a|b)^*abb$ (4 punti).

DFA: automa a stato finito deterministico

DEFINIZIONE DFA

prende in input una stringa, fa dei cambi ed in output dice se la accetta o la rifiuta
 ↓
 deve essere fatta da sinistra a destra un carattere dopo l'altro

↳ non può tornare indietro!

Un DFA è una quintupla $M = (Q, \Sigma, \delta, q_0, F)$, dove:

• Q è un insieme di stati finiti

• Σ è un alfabeto

• $\delta: Q \times \Sigma \rightarrow Q$ è una funzione di transizione

• q_0 è lo stato iniziale

• F sono gli stati finali

NFA: automa a stati finiti non deterministico

↳ in alcuni modelli di calcolo porta dei vantaggi

Un NFA è una quintupla $M = (Q, \Sigma, \delta, q_0, F)$ dove:

- Q è l'insieme degli stati
- Σ è l'alfabeto di input
- $\delta: Q \times (\Sigma \cup \{\lambda\}) \rightarrow P(Q)$ è la funzione di transizione
- q_0 è lo stato iniziale
- F è l'insieme degli stati finali

51. Dimostrare che la classe dei linguaggi regolari è chiusa rispetto all'operazione di unione $E \cup E$ tutte le altre (senza usare automi non deterministici - 4 punti).

- la classe dei linguaggi regolari è chiusa rispetto all'unione.

DIMOSTRAZIONE: dimostrazione con prodotto cartesiano

LINGUAGGI REGOLARI CHIUSA UNIONE

Supponiamo che la macchina M_1 riconosca il linguaggio regolare A_1 , dove $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ e la macchina M_2 riconosca il linguaggio regolare A_2 , dove $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$

Costruiamo M che riconosca $A_1 \cup A_2$, dove $M = (Q, \Sigma, \delta, q_0, F)$

1. $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \wedge r_2 \in Q_2\} \Rightarrow$ prodotto cartesiano di Q_1 e Q_2 ed è denotato con $Q_1 \times Q_2$

2. Σ è lo stesso di M_1 e M_2

3. Per ogni $(r_1, r_2) \in Q$ e ogni $a \in \Sigma$, sia $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$.

4. q_0 è la coppia (q_1, q_2)

5. $F = \{(r_1, r_2) \mid r_1 \in F_1 \vee r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

Questo conclude la costruzione dell'automa finito M che riconosce l'unione di A_1 e A_2 . Quindi l'unione di due linguaggi regolari è regolare e prova che la classe dei linguaggi regolari è chiusa rispetto all'operazione di unione. $\square$

52. Scrivere la definizione di automa a pila (PDA), la definizione di automa a pila deterministico (DPDA) e dimostrare che i due modelli di calcolo non riconoscono la stessa classe di linguaggi (4 punti). **QUADERNO 31 E 34 SOTTO**

Un PDA non deterministico è una settupla $M = (Q, \Sigma, \Gamma, q_0, z, F)$, dove:

- Q è l'insieme degli stati;
- Σ è l'alfabeto di input;
- Γ è l'alfabeto dello stack;
- $\delta: Q \times (\Sigma \cup \{\lambda\}) \times (\Gamma \cup \{\lambda\}) \rightarrow P(Q \times \Gamma \cup \{\lambda\})$ è la funzione di transizione;
- q_0 è lo stato iniziale;
- z è il simbolo iniziale dello stack;
- F è l'insieme degli stati finali.

L'insieme di stringhe accettate da un PDA è $L(M) = \{w : (q_0, w, z) \xrightarrow{*} (q_f, \lambda, s)\}$

Un DPDA deterministico è una settupla $M = (Q, \Sigma, \Gamma, q_0, z, F)$, dove:

- Q è l'insieme degli stati;
- Σ è l'alfabeto di input;
- Γ è l'alfabeto dello stack;
- $\delta: Q \times (\Sigma \cup \{\lambda\}) \times (\Gamma \cup \{\lambda\}) \rightarrow Q \times \Gamma \cup \{\lambda\}$ è la funzione di transizione;
- q_0 è lo stato iniziale;
- z è il simbolo iniziale dello stack;
- F è l'insieme degli stati finali.

TEOREMA: $DPDA \subset PDA \Rightarrow$ Esiste un linguaggio accettato da un PDA che non è accettato da nessun DPDA.

Consideriamo $L = \{a^n b^n\} \cup \{a^n b^{2n}\}$

1. Dimostriamo che L è context-free:

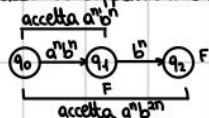
$S \rightarrow S_1 S_2$

$S_1 \rightarrow a^n b^n \Rightarrow$ è context-free

$S_2 \rightarrow a^n b^{2n}$

2. Dimostriamo che L non è context-free deterministico

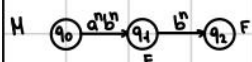
Per assurdo supponiamo che lo sia $\Rightarrow \exists DPDA$ che accetta L



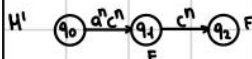
I costruttori esistono perché M è DPDA

FATTO 1: $\{a^n b^n\}$ non è context-free

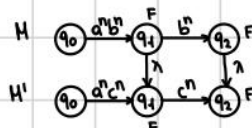
FATTO 2: $L \cup \{a^n b^n\}$ non è context-free



$\Downarrow$

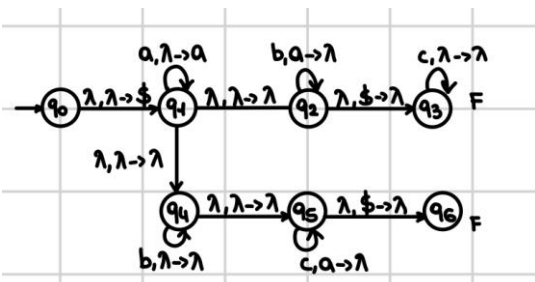


$\Rightarrow$ Costruiamo gli stati finali dei due automi:



Questo automa accetta $\{a^n b^n\} \cup \{a^n b^{2n}\} \cup \{a^n b^n\}$, ma questo è impossibile perché il linguaggio non è context-free.

53. Scrivere la definizione di automa a pila (PDA), e definire il PDA che riconosce il linguaggio $\{a^i b^j c^k \mid i, j, k \geq 0 \text{ e } i=j \text{ oppure } i=k\}$ (4 punti).



54. Dimostrare che se un linguaggio è regolare, allora esiste un'espressione regolare che lo descrive. (4 punti). [Nota bene: questa domanda è diversa dalla domanda 20. Nella 20, veniva chiesto di dimostrare la doppia implicazione L accettato da DFA $\Leftrightarrow L$ ha una regex, mentre in questo basta dimostrare l'implicazione L accettato da DFA $\Rightarrow L$ ha una regex].

2) LEMMA: Se un linguaggio è regolare, allora è descritto da un'espressione regolare.

↳ Se un linguaggio A è regolare, allora un'espressione regolare lo descrive

↳ allora è accettato da un DFA

$\Rightarrow$ Descriviamo una procedura per trasformare i DFA in espressioni regolari equivalenti

↳ la dividiamo in due parti, usando un automa finito non deterministico generalizzato, GNFA

Dobbiamo dimostrare prima come trasformare un DFA in un GNFA e poi come trasformare un GNFA in un'espressione regolare. *

↳ già anche delle transizioni possiamo avere espressioni regolari come etichette

Un GNFA è una quintupla $M = (Q, \Sigma, \delta, q_{start}, q_{accept})$, dove:

• Q è l'insieme degli stati

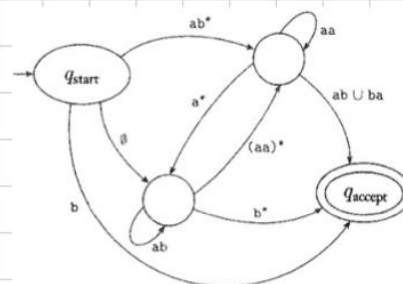
• Σ è l'alfabeto di input

• $\delta: (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$ è la funzione di transizione

• q_{start} è lo stato iniziale

• q_{accept} è lo stato finale

Esempio:



Un GNFA accetta una stringa w in Σ^* se:

• $w = w_1 w_2 \dots w_k$, con ogni w_k in Σ^*

• esiste una sequenza di stati $q_0, q_1, \dots, q_k$ tali che:

- $q_0 = q_{start}$

- $q_k = q_{accept}$

- w_i in $L(R_i)$, con $R_i = \delta(q_{i-1}, q_i)$

Condizioni per i GNFA:

• Lo stato iniziale ha anche di transizione uscenti verso un qualsiasi altro stato, ma nessun arco entrante proveniente da un qualsiasi altro stato.

• Esiste un solo stato accettante, ed esso ha anche entranti provenienti da un qualsiasi altro stato, ma nessun arco uscente. Inoltre, non è uguale a q_{start} .

• Eccetto che per lo stato iniziale e lo stato accettante, un arco va da ogni stato ad ogni altro stato ed anche ogni stato in sé stesso.

* Per trasformare un DFA in un GNFA aggiungiamo un nuovo stato iniziale con un ϵ -arco che entra nel vecchio stato iniziale ed un nuovo stato accettabile con ϵ -archi entrambi provenienti dai vecchi stati accettabili.

Se alcuni archi hanno più etichette, sostituiamo ognuno di essi con un solo arco la cui etichetta è l'unione delle precedenti etichette.

Infine, aggiungiamo anche con etichetta $\emptyset$ tra stati che non hanno archi.

Per trasformare un GNFA in un'espressione regolare, immaginiamo di supporre che il GNFA abbia k stati e sappiamo che $k \geq 2$. Se $k > 2$, costruiamo un GNFA equivalente con $k-1$ stati. Questo passo può essere ripetuto sul nuovo GNFA fino a quando esso è ridotto a due stati.

Se $k=2$, il GNFA ha un solo arco che va da q_{start} a q_{accept} . L'etichetta di questo arco è l'espressione regolare equivalente.

Per costruire un GNFA equivalente con uno stato in meno quando $k > 2$, scegliamo uno stato, estraiendolo dalla macchina, e modifichiamo il resto in modo che sia ancora riconoscuto lo stesso linguaggio. Chiamiamo q_{np} lo stato rimosso e poi modifichiamo le espressioni regolari che etichettano gli altri archi, che andranno a combinarsi come l'assemblaggio di q_{np} . Queste etichette, inoltre, costituiranno un'espressione regolare.

Nella vecchia macchina, se:

1. q_i va a q_j con un arco etichettato R_1 ,

2. q_{np} va in sé stesso con un arco etichettato R_2 ,

3. q_{np} va a q_j con un arco etichettato R_3 ,

4. q_i va a q_{np} con un arco etichettato R_4 ,

allora nella nuova macchina, l'arco da q_i a q_j riceve l'etichetta $(R_1)(R_2)^*(R_3) \cup (R_4)$. $\square$

Sia H un DFA per un linguaggio A .

Per trasformarlo in un GNFA G aggiungiamo un nuovo stato iniziale ed un nuovo stato accettabile ed i necessari archi di transizione supplementari.

Usiamo la procedura **CONVERT(G)**, che prende un GNFA e restituisce un'espressione regolare equivalente.

Usa la ricorsione, cioè chiama se stessa.

CONVERT(G):

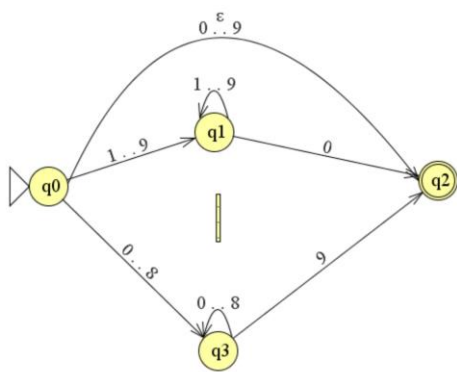
1. Sia k il numero di stati di G .

2. Se $k=2$, allora restituisce l'espressione R .

3. Se $k > 2$, scegliamo $q_{np} = q_{start} + q_{accept}$, e sia G' il GNFA $(Q', \Sigma, \delta', q_{start}, q_{accept})$, dove $Q' = Q - \{q_{np}\}$, e per ogni $q_i \in Q' - \{q_{accept}\}$ e ogni $q_j \in Q' - \{q_{start}\}$, poniamo $\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$, dove $R_1 = \delta(q_i, q_j)$, $R_2 = \delta(q_{np}, q_{np})$, $R_3 = \delta(q_{np}, q_j)$ ed $R_4 = \delta(q_i, q_{np})$.

4. Calcola **CONVERT(G')** e restituisce questo valore.

55. Definire l'automa a stati finiti che accetta parole definite su $\{0, 1, \dots, 9\}^*$ tali che la cifra finale non sia comparsa in precedenza (4 punti).



56. ANFA = { $\langle B, w \rangle \mid B$ è un NFA che accetta w } è decidibile? (4 punti).

TEOREMA: ANFA è un linguaggio decidibile.

DIMOSTRAZIONE:

Costruiamo una mdT N che decide ANFA.

Intanto tutto N conviene è NFA che riceve come input un DFA e poi lo passa a M .

Quindi: 1. Conviene è NFA B in un DFA C equivalente;

2. Esegue la mdT M su input $\langle C, w \rangle$;

3. Se M accetta, accetta; altrimenti rifiuta. $\square$

57. Dimostrare che la classe NP è chiusa rispetto all'operazione *-closure (4 punti).

58. A, B appartiene T-riconoscibile allora AUB è t-riconoscibile

59. Dimostrare l'equivalenza fra NFA e DFA (4 punti).

Due macchine si dicono equivalenti se accettano lo stesso linguaggio.

↳ NFA e DFA accettano lo stesso classe di linguaggio.

Dato un DFA che riconosce un linguaggio L , esiste corrispondentemente un NFA che riconosce lo stesso linguaggio L ; viceversa, dato un NFA che riconosce un linguaggio L' , esiste un DFA che riconosce lo stesso linguaggio L' .

DFA $\leq$ NFA (semplice)

NFA $\leq$ DFA (costruzione per sottoinsiemi - Thompson)

EQUIVALENZA NFA E DFA

DIMOSTRAZIONE:

Sia $N = (Q, \Sigma, \delta, q_0, F)$ è NFA che riconosce un linguaggio A . Vogliamo costruire un DFA $M = (Q', \Sigma, \delta', q'_0, F')$ che riconosca A , seguendo l'algoritmo di Thompson:

1. $Q' = P(Q)$, dove $P(Q)$ è l'insieme dei sottoinsiemi di Q . Ogni stato di M è l'insieme di stati di N .

2. Per $R \in Q'$ e $a \in \Sigma$, sia $\delta'(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ per qualche } r \in R\}$. Se R è uno stato di M , esso è anche un insieme di stati di N . Quando M legge un simbolo a nello stato R , mostra dove a punto ogni stato in R . Poiché da ogni stato si può andare in un insieme di stati, prendiamo l'unione di tutti questi insiemi.

3. $q'_0 = \{q_0\}$. M inizia nello stato corrispondente alla collezione che contiene solo lo stato iniziale di N .

4. $F' = \{R \in Q' \mid R \text{ contiene uno stato accettabile di } N\}$. La macchina M accetta se uno dei possibili stati in cui N potrebbe essere a quel punto è uno stato accettabile.

Ora, per ogni stato R di M , definiamo $E(R)$ come la collezione di stati che possono essere raggiunti dagli elementi di R proseguendo solo con ϵ -anchi, includendo gli stessi elementi di R . Formalmente, per $R \subseteq Q$ sia $E(R) = \{q \in Q \mid q \text{ può essere raggiunto da } R \text{ attraverso } 0 \text{ o più } \epsilon\text{-anchi}\}$.

Sostituendo $\delta'(R, a)$ con $E(\delta(R, a))$ otteniamo: $\delta'(R, a) = \{q \in Q \mid q \in E(\delta(R, a)) \text{ per qualche } r \in R\}$.

Per è necessario cambiare q'_0 in $E(q_0)$. In questo modo abbiamo completato la costruzione del DFA M che simula l'NFA N .

60. Dato il linguaggio $\{ ww \mid w \in \{0,1\}^* \}$, dimostrare che non è regolare (4 punti).

Esempio: $L = \{ ww \mid w \in \{0,1\}^* \}$

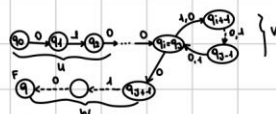
WW NON REGOLARE

Supponiamo per assurdo che L sia regolare. Allora $\exists M = (S, Q, \delta, q_0, F)$ t.c. $L = L(M)$. Sia $z = a^n b^n$ dove $|z| \geq n$, allora esiste un partizionamento $z = uvw$ t.c. $\forall uv^i w \in L$ se valgono:

1) $|uv| \leq n$

2) $v \neq \lambda$

Sia M definito come segue



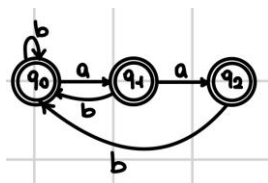
Da M deduco: $u = w_1^i, v = w_2^{j-1}, w = w_2^{n-j} w_1^n$

Considero $uv^i w = w_1^i (w_2^{j-1})^i w_2^{n-j} w_1^n = w_1^i w_2^{i(j-1)+n-j} w_1^n \Rightarrow$ devo provare che $i(n-j) \geq n$. Quindi $uv^i w \in L$

$i(n-j) \geq n \Leftrightarrow i \geq \frac{n}{n-j} \Rightarrow i \geq 1 \Rightarrow$ NO! perché $|w_1| \neq |w_2|$

Dimostro che L non è regolare

61. Scrivere il DFA che accetta il linguaggio $\{ w \in \{0,1\}^* \mid w \text{ non contiene mai più di due } 0 \text{ consecutivi} \}$ (4 punti).



62. Dimostrare che un linguaggio è regolare se e solo se esiste almeno una espressione regolare che lo descrive (4 punti). **QUADERNO PAG 19 E 20** MA RIASSUMERE

TEOREMA: Un linguaggio è regolare se e solo se è descritto da una espressione regolare.

↳ deve essere dimostrato in entrambe le direzioni

1) LEMMA: Se un linguaggio è descritto da un'espressione regolare, allora esso è regolare.

↳ Supponiamo di avere un'espressione regolare R , che descrive un linguaggio A

↳ bisogna trasformarla in un NFA che riconosca A

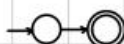
1. $R = \emptyset \Rightarrow L(R) = \emptyset$



2. $R = \epsilon \Rightarrow L(R) = \{\epsilon\}$



3. $R = a$ per qualche $a \in \Sigma \Rightarrow L(R) = \{a\}$



4. $R = R_1 \cup R_2$

5. $R = R_1 \circ R_2$

6. $R = R^*$

□

2) LEMMA: Se un linguaggio è regolare, allora è descritto da un'espressione regolare.

↳ Se un linguaggio A è regolare, allora un'espressione regolare lo descrive

↳ allora è accettato da un DFA

$\Rightarrow$ Descriviamo una procedura per trasformare i DFA in espressioni regolari equivalenti

↳ la dividiamo in due parti, usando un automa finito non deterministico generalizzato, GNFA

Dobbiamo dimostrare prima come trasformare un DFA in un GNFA e poi come trasformare un GNFA in un'espressione regolare. *

↳ già anche delle transizioni possiamo avere espressioni regolari come etichette

Un GNFA è una quintupla $M = (Q, \Sigma, \delta, q_{start}, q_{accept})$, dove:

• Q è l'insieme degli stati

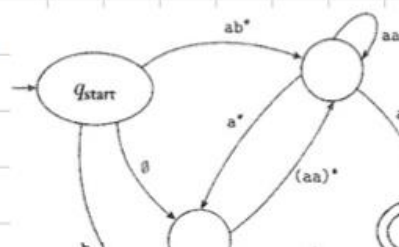
• Σ è l'alfabeto di input

• $\delta: (Q - \{q_{accept}\}) \times (Q - \{q_{start}\}) \rightarrow R$ è la funzione di transizione

• q_{start} è lo stato iniziale

• q_{accept} è lo stato finale

Esempio:



* Per trasformare un DFA in un GNFA aggiungiamo un nuovo stato iniziale con un ϵ -arco che entra nel vecchio stato iniziale ed un nuovo stato accettabile con ϵ -archi entrambi provenienti dai vecchi stati accettabili.

Se alcuni archi hanno più etichette, sostituiamo ognuno di essi con un solo arco la cui etichetta è l'unione delle precedenti etichette.

Infine, aggiungiamo archi con etichetta $\emptyset$ tra stati che non hanno archi.

Per trasformare un GNFA in un'espressione regolare, immaginiamo di supporre che il GNFA abbia k stati e sappiamo che $k \geq 2$. Se $k > 2$, costruiamo un GNFA equivalente con $k-1$ stati. Questo passo può essere ripetuto sul nuovo GNFA fino a quando esso è ridotto a due stati.

Se $k=2$, il GNFA ha un solo arco che va da q_{start} a q_{accept} . L'etichetta di questo arco è l'espressione regolare equivalente.

Per costruire un GNFA equivalente con uno stato in meno quando $k > 2$, scegliamo uno stato, estraiendolo dalla macchina, e modifichiamo il resto in modo che sia ancora riconoscuto lo stesso linguaggio. Chiamiamo q_{np} lo stato rimosso e poi modifichiamo le espressioni regolari che etichettano gli altri archi, che andranno a combinatele con l'assembla di q_{np} . Queste etichette, inoltre, costituiscono un'espressione regolare.

Nella vecchia macchina, se:

1. q_i va a q_j con un arco etichettato R_1 ,

2. q_{np} va in sé stesso con un arco etichettato R_2 ,

3. q_{np} va a q_j con un arco etichettato R_3 ,

4. q_i va a q_j con un arco etichettato R_4 ,

allora nella nuova macchina, l'arco da q_i a q_j riceve l'etichetta $(R_1)(R_2)^*(R_3) \cup (R_4)$. $\square$

Sia M un DFA per un linguaggio A .

Per trasformarlo in un GNFA G aggiungiamo un nuovo stato iniziale ed un nuovo stato accettabile ed i necessari archi di transizione supplementari.

Usiamo la procedura $CONVERT(G)$, che prende un GNFA e restituisce un'espressione regolare equivalente.

Usa la ricorsione, cioè chiama se stesso.

$CONVERT(G)$:

1. Sia k il numero di stati di G .

2. Se $k=2$, allora restituisce l'espressione R .

3. Se $k > 2$, scegliamo $q_{np} \neq q_{start} \neq q_{accept}$, e sia G' il GNFA $(Q', \Sigma, \delta', q_{start}, q_{accept})$, dove $Q' = Q - \{q_{np}\}$, e per ogni $q_i \in Q' - \{q_{accept}\}$ e ogni $q_j \in Q' - \{q_{start}\}$, poniamo $\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$, dove $R_1 = \delta(q_i, q_j)$, $R_2 = \delta(q_{np}, q_{np})$, $R_3 = \delta(q_{np}, q_j)$ ed $R_4 = \delta(q_i, q_j)$.

4. Allora $CONVERT(G')$ e restituisce questo valore.

63. Dimostrare che NP è chiusa rispetto all'operazione star *

7.43 Si mostri che NP è chiusa rispetto all'operazione star.

DIMOSTRAZIONE

7.43 Sia $A \in NP$. Si costruisce una NTM M per decidere A^* in tempo polinomiale non deterministico.

$M =$ "Su input w :

1. Dividi non deterministicamente w in parti $w = x_1 x_2 \dots x_k$.
2. Per ciascun x_i , ipotizza non deterministicamente il certificato che mostra che $x_i \in A$.
3. Verifica tutti i certificati se possibile, poi accetta. Altrimenti, se la verifica fallisce, rifiuta."

64. Siano B e C due linguaggi $\Sigma=\{0,1\}$. Definiamo $B \leftarrow C = \{w \in B \mid \text{per qualche } y \in C, \text{ le stringhe } w \text{ e } y \text{ hanno lo stesso numero di } 1\}$. Mostrare che la classe dei linguaggi regolari è chiusa rispetto all'operazione $\leftarrow$.

uguale.

1.34 Siano $M_B = (Q_B, \Sigma, \delta_B, q_B, F_B)$ ed $M_C = (Q_C, \Sigma, \delta_C, q_C, F_C)$ due DFA che riconoscono B e C , rispettivamente. Costruiamo un NFA $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce $B \leftarrow C$ come segue. Per decidere se il suo input w è in $B \leftarrow C$, la macchina M verifica che $w \in B$, e in parallelo non deterministicamente ipotizza una stringa y che contiene lo stesso numero di simboli uguali a 1 di w e verifica che $y \in C$.

1. $Q = Q_B \times Q_C$.

2. Per $(q, r) \in Q$ e $a \in \Sigma_\epsilon$, definiamo

$$\delta((q, r), a) = \begin{cases} \{(\delta_B(q, 0), r)\} & \text{se } a = 0 \\ \{(\delta_B(q, 1), \delta_C(r, 1))\} & \text{se } a = 1 \\ \{(q, \delta_C(r, 0))\} & \text{se } a = \epsilon. \end{cases}$$

3. $q_0 = (q_B, q_C)$.

4. $F = F_B \times F_C$.

MDT universale.

La mdt universale è una mdt riprogrammabile, cioè che può eseguire ogni altra mdt.

Useremo la codifica della mdt in unario, dove ogni transizione è una quintupla $((q, a) = (p, b, m))$.

Tra gli elementi della quintupla ci sono degli elementi separatori (0), e anche tra le varie quintuple (00).

Definita la codifica, definiamo la mdt U come una mdt a 4 nastri:

- 1- Contiene $\text{des}(M)$, ovvero la codifica
- 2- Contiene l'input x di M
- 3- Memorizza lo stato di M in ogni istante
- 4- Contiene lo stato di accettazione di M

U opera come segue:

- 1- Nello stato q_0 :
 - Copia w_0 in N_3 e w_1 in N_4

- Sposta la testina di N1 sul simbolo a destra del primo $\otimes$ che incontra
 - Va in q1
- 2- Nello stato q1, cerca su N1 la quintupla che abbia come primo simbolo lo stesso simbolo letto da N3 e come secondo simbolo lo stesso letto da N2, cioè la transizione che M dovrebbe eseguire in quello stato leggendo quell'input. Se la trova, passa in q_scrivi. Se arriva a $\square$ senza trovarla, verifica che M si trovi in uno stato di accettazione, confrontando i simboli letti su N3 e N4. Se sono uguali, accetta, altrimenti rifiuta.
 - 3- Nello stato q_scrivi, U scrive il terzo carattere della quintupla trovata in N1 su N2, e si sposta su q_cambiastato.
 - 4- Nello stato q_cambiastato, U scrive il quarto carattere della quintupla trovata in N1 su N3, e si sposta in q_muovi
 - 5- Nello stato q_muovi, U muove la testina di N2 in base al carattere letto nell'ultimo elemento della quintupla (lo spostamento della testina), e si sposta in q_riavvolgi
 - 6- Nello stato q_riavvolgi, U sposta la testina del nastro N1 all'inizio delle transizioni, cioè dopo il primo $\otimes$ che incontra.

Teorema di savitch. (PSPACE = NPSPACE)

Il teorema di savitch dice che per ogni $f: \mathbb{N} \rightarrow \mathbb{R}^+$, con $f(n) \geq n$,

$\text{NPSPACE}(f(n)) \subseteq \text{PSPACE}(f^2(n))$, ovvero che nello spazio la complessità della simulazione da “non determinismo” a “determinismo” aumenta in modo al più quadratico.

Siano date due configurazioni c_1 e c_2 di una NTM N e un numero t .

Vogliamo verificare se N può raggiungere c_2 da c_1 in al più t passi (problema della resa).

Questo problema può essere risolto da una mdt deterministica.

Sia N una NTM che decide il linguaggio A con spazio $O(f(n))$.

Costruiamo una mdt M che decide A .

M userà un'altra mdt “canyeld” per verificare se da una configurazione ne può raggiungere un'altra in un numero di passi prefissato.

Nello specifico:

canyeld, su input c_1 , c_2 e t

- Se $t=1$, verifica se $c1 = c2$ oppure se $c2$ è raggiungibile con un passo. Se sì, accetta, altrimenti rifiuta.
- Se $t > 1$, per ogni configurazione cm di N su w :
 - a. Esegue $\text{canyeld}(c1, cm, t/2)$, esegue $\text{canyeld}(cm, c2, t/2)$
 - b. Se entrambe accettano allora accetta, altrimenti rifiuta.

Modifichiamo N in modo che abbia una sola configurazione accettante cf .

Sia $c0$ la configurazione iniziale di N su w .

Scegliamo una costante d tale che N non abbia più di $2^{df(n)}$ configurazioni.

$2^{df(n)}$ limita superiormente il tempo di esecuzione di ogni diramazione di N .

A questo punto, M , su input w , fornisce il risultato di $\text{CANYELD}(c0, cf, 2^{df(n)})$.

Ogni qualvolta canyeld invoca se stessa ricorsivamente, memorizza t , $c1$, $c2$ in una pila, in modo tale che questi valori possono poi essere recuperati. Ciascun livello della ricorsione usa uno spazio pari a $O(f(n))$.

Successivamente, ad ogni ricorsione, t viene diviso per 2. Essendo il primo valore di $t = 2^{df(n)}$, la profondità della ricorsione è $O(\log(2^{df(n)}))$, ovvero $O(f(n))$. Quindi lo spazio usato in totale è $O(f^2(n))$.

È importante però che M , quando esegue CANYELD , conosca il valore di $f(n)$.

Per risolvere questo possiamo modificare M in modo che tenti i valori di $f(n)$ ad uno ad uno: 1, 2, 3,....

Per ciascun valore $f(n) = i$, quindi, M' tenterà CANYELD per vedere se la configurazione accettante è raggiungibile.

Se essa è raggiungibile, M' accetta, altrimenti controlla se ci siano configurazioni che usano più di i celle. Se non ce ne sono, si ferma e rifiuta, in quanto non ha senso continuare per $i+1$ visto che non ci sono configurazioni, e si evitano sprechi di spazio.

TQBF P-SPACE COMPLETO.

Per dimostrare che è PSPACE completo, dobbiamo prima dimostrare che è in PSPACE, e poi dimostrare che è PSPACE hard, ovvero che ogni problema in PSPACE è riducibile a TQBF.

Tqbf è in pspace.

TQBF = $\{ \phi \mid \phi \text{ è una formula booleana chiusa vera} \}$

TEOREMA: TQBF è PSPACE-completo.

DIMOSTRAZIONE:

$T =$ "Su input $\langle \phi \rangle$, una formula booleana compattamente quantificata:

1. Se ϕ non ha quantificatori, valutare ed eventualmente accettare;
2. Se $\phi = \exists x \psi$, chiama T su ψ con 0 (e poi 1) al posto di x ; se uno dei due accetta, allora accetta; altrimenti rifiuta;
3. Se $\phi = \forall x \psi$, chiama T su ψ con 0 (e poi 1) al posto di x ; se tutti e due accettano, allora accetta; altrimenti rifiuta."

L'algoritmo T decide TQBF. Lo spazio totale utilizzato è $O(m)$, dove m è il numero di variabili presenti in ϕ .

TQBF è PSPACE-hard.

Sia A un linguaggio deciso da una mdt M in spazio n^k per qualche costante k . Costruisco un tableau per la computazione di M su w , ed una formula ϕ che esprime un tableau accettabile. Il tableau è lungo $O(n^k)$, ma è profondo $2^{O(n)}$.

Se c_1 e c_2 sono due configurazioni di variabili, la formula $\phi_{c_1, c_2, t}$ è vera se e solo se M può andare da c_1 a c_2 in al più t passi.

Allora possiamo definire ϕ come la formula $\phi_{start, accept, h}$, dove $h = 2^{O(n)}$ per una costante O , scelta in modo tale che M abbia non più di $2^{O(n)}$ configurazioni possibili su

un input di lunghezza n .

La formula codifica i contenuti delle celle di una configurazione come nella prova del teorema di Cook e Levin.

Se $t=1$, costruisco $\phi_{c_1, c_2, 1}$ ($c_1 = c_2$) oppure vado da c_1 a c_2 in un passo.

Se $t>1$, $\exists m_1 [\phi_{c_1, m_1, t-1} \wedge \phi_{m_1, c_2, t-1}]$.

Per ridurre la dimensione della formula, usiamo $\forall$ in aggiunta a $\exists$. Sia $\phi_{c_1, c_2, t} = \exists m_1 \forall (c_3, c_4) [f(c_1, m_1, m_1, c_2) \wedge \phi_{c_3, c_4, t-1}]$.

Per calcolare la dimensione della formula $\phi_{start, accept, h}$ dove $h = 2^{O(n)}$, notiamo che ciascun livello della ricorsione aggiunge una porzione della formula che è lineare nella dimensione della configurazione, ed è perciò di dimensione $O(n)$. Il numero di livelli della ricorsione è $\log(2^{O(n)})$, cioè $O(n)$. Quindi la dimensione della formula risultante è $O(n^2)$. $\square$

Costruiamo un algoritmo T che, su input $\langle \phi \rangle$:

- Se ϕ non contiene quantificatori, allora T valuta ϕ e accetta se è vera, e rifiuta se è falsa.
- Se ϕ è uguale a $\exists x \psi$, invoca ricorsivamente T su ψ , prima con 0 al posto di x e poi con 1. Se almeno uno dei risultati è di accettazione, accetta, se entrambi non lo sono, rifiuta.
- Se ϕ è uguale a $\forall x \psi$, invoca ricorsivamente T su ψ , prima con 0 al posto di x e poi con 1. Se entrambi i risultati sono di accettazione, accetta, se anche uno non lo è, rifiuta.

La complessità in spazio dipende dal numero di variabili, quindi ha complessità $O(m)$, dove m è appunto il numero di variabili.

Tqbf è pspace hard.

Sia A un linguaggio deciso da una mdt M in spazio n^k per qualche costante k .

La riduzione associa una stringa w ad una formula booleana quantificata ϕ che è vera se M accetta w .

Per costruire ϕ , usiamo due collezioni di variabili c_1 e c_2 , che rappresentano due configurazioni, e un numero $t > 0$.

Se c_1 e c_2 sono configurazioni reali, la formula $\phi_{c_1, c_2, t}$ è vera se M può andare da c_1 a c_2 in massimo t passi.

Allora definiamo ϕ come la formula $\phi_{c_{start}, c_{accept}, h}$, dove $h = 2^{df(n)}$, con d costante scelta in modo tale che M non abbia più di $2^{df(n)}$ configurazioni. Sia $f(n) = n^k$.

- Se $t = 1$, progettiamo la formula per dire che $c_1 = c_2$ oppure da c_1 vado a c_2 con un passo.
- Se $t > 1$, costruiamo $\phi_{c_1, c_2, t}$ ricorsivamente. Possiamo pensare inizialmente di costruirla in questo modo:

$\phi_{c_1, c_2, t} = \exists m_1 [\phi_{c_1, m_1, t/2} \wedge \phi_{m_1, c_2, t/2}]$, dove m_1 è una config intermedia di M .

Qui però abbiamo un problema in quanto ogni livello della ricorsione dimezza t ma raddoppia la dimensione della formula. Quindi verrà prodotta una formula di dimensione esponenziale.

Per ridurre la dimensione, useremo il quantificatore $\forall$:

$\phi_{c_1, c_2, t} = \exists m_1 \forall (c_3, c_4) \in \{(c_1, m_1), (m_1, c_2)\} [\phi_{c_3, c_4, t/2}]$.

Si tratta comunque di una formula ricorsiva in cui:

- Lo spazio usato da ogni ricorsione è lineare alla dimensione della configurazione, quindi $O(f(n))$
- Il numero di ricorsioni è pari a $\log(2^{df(n)})$, quindi $O(f(n))$.

Ne deriva che la complessità della formula è $O(f^2(n))$ che è polinomiale.

Pumping lemma per i linguaggi regolari

Poiché x porta M da r_1 a r_j , y porta M da r_j a r_j e z porta M da r_j a r_{n+1} , che è uno stato accetante, M deve accettare xy^iz per $i \geq 0$. Sappiamo che $j \neq l$, perciò $|y| > 0$; e $l \leq p+1$, perciò $|xy| \leq p$. Quindi tutte le condizioni del pumping lemma sono rispettate.

DIMOSTRAZIONE Sia $M = (Q, \Sigma, \delta, q_1, F)$ un DFA che riconosce A e sia p il numero di stati di M .

Sia $s = s_1 s_2 \dots s_n$ una stringa in A di lunghezza n , dove $n \geq p$. Sia $r_1, \dots, r_{n+1}$ la sequenza di stati attraversati da M mentre elabora s , quindi $r_{i+1} = \delta(r_i, s_i)$ per $1 \leq i \leq n$. Questa sequenza ha lunghezza $n+1$, che è almeno $p+1$. Due tra i primi $p+1$ elementi nella sequenza devono essere lo stesso stato, per il principio della piccionaia. Chiamiamo il primo di questi r_j e il secondo r_l . Poiché r_l si presenta tra le prime $p+1$ posizioni in una sequenza che inizia in r_1 , abbiamo $l \leq p+1$. Ora sia $x = s_1 \dots s_{j-1}$, $y = s_j \dots s_{l-1}$ e $z = s_l \dots s_n$.

Per usare il pumping lemma per provare che un linguaggio B non è regolare, in primo luogo si assume che B sia regolare per ottenere una contraddizione. Poi si usi il pumping lemma per assicurare l'esistenza di una lunghezza del pumping p tale che tutte le stringhe di lunghezza maggiore o uguale a p in B possano essere iterate. In seguito, si trovi una stringa s in B che ha lunghezza maggiore o uguale a p , ma che non può essere iterata. Infine, si dimostri che s non può essere iterata considerando tutti i modi di dividere s in x, y e z (prendendo in considerazione la condizione 3 del pumping lemma se è utile) e, per ogni tale divisione, trovando un valore i tale che $xy^iz \notin B$. Questo passo finale spesso comporta il dover raggruppare i vari modi di dividere s in diversi casi e l'analizzarli individualmente. L'esistenza di s contraddirebbe il pumping lemma se B fosse regolare. Quindi B non può essere regolare.

Trovare s a volte richiede un po' di ragionamento creativo. Potresti dover cercare tra diversi candidati per s prima di scoprirne uno che funzioni. Prova con elementi di B che sembrano esibire l'"essenza" della non regolarità di B . Discutiamo ulteriormente il compito di trovare s in alcuni degli esempi seguenti.

2. Per $(q, r) \in Q \times \Sigma$ e $a \in \Sigma$, definiamo

$$\delta((q, r), a) = \begin{cases} \{\delta_B(q, 0), r\} & \text{se } a = 0 \\ \{\delta_B(q, 1), \delta_C(r, 1)\} & \text{se } a = 1 \\ \{(q, \delta_C(r, 0))\} & \text{se } a = \epsilon. \end{cases}$$

3. $q_0 = (q_B, q_C)$.

4. $F = F_B \times F_C$.

Se un linguaggio è C.F. allora esiste un automa a pila che lo riconosce

DIMOSTRAZIONE Ora diamo i dettagli formali della costruzione dell'automa a pila $P = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, F)$. Per rendere più chiara la costruzione, usiamo una notazione abbreviata per la funzione di transizione. Questa notazione fornisce un modo per scrivere un'intera stringa sulla pila in un passo della macchina. Possiamo simulare quest'azione introducendo stati aggiuntivi per scrivere la stringa un simbolo alla volta, come realizzato nella seguente costruzione formale.

Siano q ed r stati del PDA e siano a in Σ ed s in Γ^* . Supponiamo di volere che il PDA vada da q a r quando legge a ed elimina s . Inoltre, vogliamo che esso inserisca l'intera stringa $u = u_1 \dots u_l$ sulla pila simultaneamente. Possiamo eseguire questa azione introducendo nuovi stati $q_1, \dots, q_{l-1}$ e definendo la funzione di transizione come segue:

$$\begin{aligned} \delta(q, a, s) &\text{ contiene } (q_1, u_l), \\ \delta(q_1, \epsilon, \epsilon) &= \{(q_2, u_{l-1})\}, \\ \delta(q_2, \epsilon, \epsilon) &= \{(q_3, u_{l-2})\}, \\ &\vdots \\ \delta(q_{l-1}, \epsilon, \epsilon) &= \{(r, u_1)\}. \end{aligned}$$

Useremo la notazione $(r, u) \in \delta(q, a, s)$ per denotare che quando q è lo stato in cui si trova l'automa, a è il prossimo simbolo di input ed s è il simbolo sulla cima della pila, il PDA può leggere a ed eliminare s , poi inserire la stringa u nella pila e passare nello stato r . La figura seguente ne mostra la realizzazione.



FIGURA 2.23 Implementazione dell'abbreviazione $(r, xyz) \in \delta(q, a, s)$

Gli stati di P sono $Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{accept}}\} \cup E$, dove E è l'insieme degli stati ausiliari per realizzare l'abbreviazione appena descritta. Lo stato iniziale è q_{start} . L'unico stato accetante è q_{accept} .

La funzione di transizione è definita come segue. Cominciamo inizializzando la pila inserendo i simboli $\$$ and S , realizzando così il passo 1 nella descrizione informale: $\delta(q_{\text{start}}, \epsilon, \epsilon) = \{(q_{\text{loop}}, S\$)\}$. Poi aggiungiamo le transizioni per il ciclo principale del passo 2.

In primo luogo, trattiamo il caso (a) in cui la cima della pila contiene una variabile. Poniamo $\delta(q_{\text{loop}}, \epsilon, A) = \{(q_{\text{loop}}, w)\}$ dove $A \rightarrow w$ è una regola in R .

In secondo luogo, trattiamo il caso (b) in cui la cima della pila contiene un terminale. Poniamo $\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \epsilon)\}$.

Infine, trattiamo il caso (c) in cui il marcatore scelto per indicare la pila vuota $\$$ è sulla cima della pila. Poniamo $\delta(q_{\text{loop}}, \epsilon, \$) = \{(q_{\text{accept}}, \epsilon)\}$. Il diagramma di stato è mostrato nella Figura 2.24.



FIGURA 2.24 Diagramma di stato di P

Pumping lemma per i linguaggi C.F.

Il pumping lemma per i linguaggi context-free Se A è un linguaggio context-free, allora esiste un numero p (la lunghezza del pumping) tale che, se s è una qualsiasi stringa in A di lunghezza almeno p , allora s può essere divisa in cinque parti $s = uvxyz$ che soddisfano le condizioni

1. per ogni $i \geq 0$, $uv^i xy^i z \in A$,
2. $|vy| > 0$ e
3. $|vxy| \leq p$.

Quando s è divisa in $uvxyz$, la condizione 2 afferma che v oppure y non è la stringa vuota. Altrimenti il teorema sarebbe banalmente vero. La condizione 3 afferma che le parti v , x e y insieme hanno lunghezza al più p . Questa condizione tecnica a volte è utile per dimostrare che alcuni linguaggi non sono context-free.

DIMOSTRAZIONE Sia G una CFG per il CFL A . Sia b il massimo numero di simboli nel lato destro di una regola (assumiamo che sia almeno 2). Sappiamo che, in ogni albero sintattico costruito usando questa grammatica, un nodo non può avere più di b figli. In altre parole, ci sono al più b foglie in un passo dalla variabile iniziale; e ci sono al più b^2 foglie in 2 passi dalla variabile iniziale; e ci sono al più b^h foglie in h passi dalla variabile iniziale. Quindi, se l'altezza dell'albero sintattico è al più h , la lunghezza della stringa generata è al più b^h . Viceversa, se una stringa generata ha lunghezza maggiore o uguale a $b^h + 1$, ciascuno dei suoi alberi sintattici deve avere un'altezza maggiore o uguale a $h + 1$.

Sia $|V|$ il numero delle variabili in G . Poniamo p , la lunghezza del pumping, uguale a $b^{|V|+1}$. Ora se s è una stringa in A e la sua lunghezza è

maggiore o uguale a p , il suo albero sintattico deve avere altezza maggiore o uguale a $|V| + 1$, poiché $b^{|V|+1} \geq b^{|V|} + 1$.

Per vedere come iterare una tale stringa s , sia τ uno dei suoi alberi sintattici. Se s ha diversi alberi sintattici, scegliamo un albero sintattico τ che abbia il più piccolo numero di nodi. Sappiamo che τ deve avere altezza maggiore o uguale a $|V| + 1$, quindi il suo cammino più lungo dalla radice a una foglia ha lunghezza almeno $|V| + 1$. Questo cammino ha almeno $|V| + 2$ nodi; uno etichettato da un terminale, gli altri etichettati da variabili. Quindi questo cammino ha almeno $|V| + 1$ variabili. Poiché G ha solo $|V|$ variabili, qualche variabile R è presente più di una volta su questo cammino. Per un utilizzo successivo, scegliamo R in modo che sia una variabile che si ripete tra le $|V| + 1$ variabili più in basso su questo cammino.

Dividiamo s in $uvxyz$ come nella Figura 2.35. Ogni occorrenza di R ha un sottoalbero sotto essa che genera una parte della stringa s . L'occorrenza più in alto di R ha un sottoalbero più grande e genera vxy , mentre l'occorrenza più in basso genera solo x con un sottoalbero più piccolo. Entrambi questi sottoalberi sono generati dalla stessa variabile, quindi possiamo sostituire l'uno con l'altro e ottenere ancora un albero sintattico corretto. Sostituire ripetutamente il più piccolo con il più grande fornisce gli alberi sintattici per le stringhe $uv^i xy^i z$ per ogni $i > 1$. Sostituire il più grande con il più piccolo genera la stringa uxz . Questo dimostra la condizione 1 del lemma. Ora veniamo alle condizioni 2 e 3.

Per ottenere la condizione 2, dobbiamo essere sicuri che v e y non sono entrambe ϵ . Se lo fossero, l'albero sintattico ottenuto sostituendo il più piccolo sottoalbero al più grande avrebbe meno nodi di τ e genererebbe ancora s . Questo non è possibile perché abbiamo scelto τ in modo che sia un albero sintattico per s con il più piccolo numero di nodi. Questa è la ragione per aver selezionato così τ .

Per ottenere la condizione 3, dobbiamo essere sicuri che vxy ha lunghezza al più p . Nell'albero sintattico per s l'occorrenza più in alto di R genera vxy . Abbiamo scelto R in modo che entrambe le occorrenze di essa cadano nelle $|V| + 1$ variabili più in basso del cammino e abbiamo scelto il più lungo cammino nell'albero sintattico, in modo che il sottoalbero in cui R genera vxy sia alto al più $|V| + 1$. Un albero con questa altezza può generare una stringa di lunghezza al più $b^{|V|+1} = p$.

TEOREMA

2.50 Fai riferimento al Problema 1.31 per la definizione dell'operazione di shuffle perfetto. Mostra che la classe dei linguaggi context-free non è chiusa rispetto allo shuffle perfetto.

DIMOSTRAZIONE

2.50 Sia A il linguaggio $\{0^k 1^k \mid k \geq 0\}$ e sia B il linguaggio $\{a^k b^{3k} \mid k \geq 0\}$. Lo shuffle perfetto di A è B è il linguaggio $C = \{(0a)^k (0b)^k (1b)^{2k} \mid k \geq 0\}$. È facile vedere che i linguaggi A e B sono CFL ma C non è un CFL, come vedremo. Se C fosse

un CFL, sia p la lunghezza del pumping data dal pumping lemma e sia s la stringa $(0a)^p (0b)^p (1b)^{2p}$. Poiché s è più lunga di p ed $s \in C$, possiamo dividere $s = uvxyz$ in modo da soddisfare le tre condizioni del pumping lemma. Le stringhe in C hanno esattamente un quarto di simboli uguali a 1 e un ottavo di simboli uguali ad a . Affinché $uv^2 xy^2 z$ abbia questa proprietà, la stringa vxy deve contenere sia simboli uguali a 1 che ad a . Ciò è impossibile perché i simboli uguali a 1 e le a sono separati da $2p$ simboli in s , ma la terza condizione afferma che $|vxy| \leq p$. Quindi C non è context-free.

TEOREMA**3.15** Mostrare che la classe dei linguaggi Turing-riconoscibili è chiusa rispetto a

- | | |
|--------------------|------------------|
| a. unione. | d. intersezione. |
| b. concatenazione. | e. omomorfismo. |
| c. star. | |

DIMOSTRAZIONE

3.15 Per ogni coppia di linguaggi decidibili L_1 e L_2 , siano M_1 e M_2 le TM che li decidono. Costruiamo una TM M' che decide l'unione di L_1 e L_2 :

“Su input w :

1. Esegue M_1 e M_2 alternativamente su w passo dopo passo. Se una accetta, accetta. Se entrambe si fermano e rifiutano, rifiuta.”

Se M_1 accetta w oppure M_2 accetta w , allora M' accetta w perché la TM che accetta arriva al suo stato-accetta dopo un numero finito di passi. Si noti che se entrambe M_1 e M_2 rifiutano e una va in loop, allora anche M' andrà in loop.

TEOREMA 3.21

Un linguaggio è Turing-riconoscibile se e solo se esiste un enumeratore che lo enumera.

DIMOSTRAZIONE. Come prima cosa, dimostriamo che se abbiamo un enumeratore E che enumera un linguaggio A , allora esiste una TM M che riconosce A . La TM M funziona come segue.

M = “Su input w :

1. Esegue E . Ogni volta che E genera una stringa, la confronta con w .
2. Se w appare nell'output di E , accetta.”

Chiaramente, M accetta quelle stringhe che compaiono sulla lista di E .

Ora occupiamoci dell'altra direzione. Se la TM M riconosce un linguaggio A , possiamo costruire il seguente enumeratore E per A . Sia $s_1, s_2, s_3, \dots$ una lista di tutte possibili stringhe in Σ^* .

E = “Ignora l'input.

1. Ripete i seguenti passi per $i = 1, 2, 3, \dots$
2. Esegue M per i passi su ogni input, $s_1, s_2, \dots, s_i$.
3. Se una qualche computazione accetta, stampa la corrispondente s_j .”

Se M accetta una particolare stringa s , alla fine s apparirà nella lista generata da E . In realtà, essa apparirà nella lista un numero infinito di volte, perché M ritorna all'inizio su ogni stringa per ogni ripetizione del passo 1.

Questa procedura fa sì che M lavori in parallelo su tutte le possibili stringhe di input.

TEOREMA: Un linguaggio è decidibile se e solo se esiste una qualche macchina di Turing non deterministica che lo decide.

3.3 Dimostriamo entrambe le direzioni del sse. In primo luogo, se un linguaggio L è decidibile, esso può essere deciso da una macchina di Turing deterministica, che è automaticamente una macchina di Turing non deterministica. In secondo luogo, se un linguaggio L è deciso da una TM N non deterministica, modifichiamo la TM D deterministica descritta nella dimostrazione del Teorema 3.16 come segue.

Rendiamo la fase 4 come fase 5.

Aggiungiamo una nuova fase 4: Rifiuta se tutti i cammini del non determinismo di N hanno portato ad un rifiuto.

Possiamo dedurre che la nuova TM D' è un decisore per L . Se N accetta il suo input, allora D' troverà un cammino che porta all'accettazione ed accetta. Se N rifiuta il suo input, tutti i suoi cammini terminano e rifiutano perché è un decisore. Quindi ciascuno dei cammini ha un numero finito di nodi, dove ogni nodo rappresenta un passo di computazione di N , lungo quel cammino. Pertanto l'intero albero di

computazione di N su questo input è finito, in virtù del teorema sugli alberi fornito nella traccia dell'esercizio. Quindi, D' si fermerà e rifiuterà quando l'intero albero sarà stato esplorato.

TEOREMA

^A3.17 Chiamiamo *macchina di Turing a sola scrittura* una TM a nastro singolo che può modificare ogni cella del nastro al più una volta (inclusa la parte di input del nastro). Mostrare che questa variante di macchina di Turing è equivalente alla macchina di Turing usuale. (Suggerimento: Come primo passo considerare il caso in cui la macchina di Turing può modificare ogni posizione del nastro al massimo due volte. Usa molto nastro.)

DIMOSTRAZIONE

3.17 In primo luogo simuliamo una macchina di Turing mediante una macchina di Turing a doppia scrittura. La macchina di Turing a doppia scrittura simula un singolo passo della macchina originaria copiando l'intero nastro su una porzione nuova di nastro al lato destro della parte attualmente utilizzata. L'operazione di copia procede carattere per carattere, marcando un carattere appena esso viene copiato. Questa procedura modifica ogni cella del nastro due volte: una volta per scrivere il carattere per la prima volta e una seconda volta per marcare che è stato copiato. La posizione della testina della macchina di Turing originale è segnata sul nastro. Quando si copiano le celle corrispondenti alla posizione marcata o adiacenti a tale

posizione, il contenuto del nastro viene aggiornato in base alle regole della macchina di Turing originale.

Per effettuare la simulazione con una macchina a sola scrittura, si può operare come prima, tranne che qui ogni cella del nastro precedente viene ora rappresentata mediante due celle. La prima contiene il simbolo della macchina originaria e la seconda è dedicata alla marcatura utilizzata nella procedura di copia. L'input non è presentato alla macchina nel formato con due celle per simbolo, così la prima volta che il nastro viene copiato, i marcatori di copiatura sono messi direttamente sopra i simboli in input.

TEOREMA

^A5.26 Considerare il problema di determinare se una macchina di Turing a due nastri eventualmente scrive un simbolo diverso dal blank sul suo secondo nastro quando funziona sull'input w . Formulare questo problema come un linguaggio e mostrare che esso è indecidibile.

DIMOSTRAZIONE

5.26 Sia $B = \{\langle M, w \rangle \mid M \text{ una TM a due nastri che scrive un simbolo diverso dal blank sul suo secondo nastro quando funziona su input } w\}$. Mostriamo che esiste una riduzione da A_{TM} a B . Supponiamo per assurdo che TM R decida B . Possiamo costruire una TM S che usa R per decidere A_{TM} .

$S =$ "Su input $\langle M, w \rangle$:

1. Usa M per costruire la seguente TM T a due nastri.

$T =$ "Su input x :

1. Simula M su x usando il primo nastro.
2. Se la simulazione mostra che M accetta, scrive un simbolo diverso dal blank sul secondo nastro."
2. Esegue R su $\langle T, w \rangle$ per determinare se T su input w scrive un simbolo diverso dal blank sul secondo nastro.
3. Se R accetta, M accetta w , quindi *accetta*. Altrimenti, *rifiuta*."

Ogni linguaggio context-free è un elemento di P.

IDEA. Nel Teorema 4.9, abbiamo dimostrato che ogni CFL è decidibile. Per fare ciò abbiamo dato un algoritmo che lo decide. Se questo algoritmo venisse eseguito in tempo polinomiale, questo teorema seguirebbe come corollario. Ricapitoliamo l'algoritmo e vediamo se funziona abbastanza velocemente. Sia L un CFL generato da una CFG G in forma normale di Chomsky. Dal Problema 2.38, qualsiasi derivazione di una stringa w ha $2n - 1$ passi, dove n è la lunghezza di w perché G è in forma normale di Chomsky. Il decisore per L funziona provando tutte le possibili derivazioni con $2n - 1$ passi quando il suo input è una stringa di lunghezza n . Se uno di questi è una derivazione di w , il decisore accetta, altrimenti, rifiuta.

Una rapida analisi di questo algoritmo dimostra che non viene eseguito in tempo polinomiale. Il numero di derivazioni con k passi può essere esponenziale in k , per cui questo algoritmo può richiedere tempo esponenziale.

Per ottenere un algoritmo di tempo polinomiale introduciamo una potente tecnica chiamata **programmazione dinamica**. Questa tecnica utilizza l'accumulo di informazioni su sottoproblemi più piccoli per risolvere problemi più grandi. Memorizziamo la soluzione ad ogni sottoproblema in modo da doverlo risolvere solo una volta. Lo facciamo creando una tabella di tutti i sottoproblemi e inserendo le loro soluzioni sistematicamente appena le troviamo. Nel caso in oggetto, consideriamo i sottoproblemi consistenti nel determinare se ogni variabile in G genera ciascuna sottostringa di w . L'algoritmo inserisce la soluzione a questo sottoproblema in una tabella $n \times n$. Per $i \leq j$, la voce (i, j) -esima della tabella contiene la collezione di variabili che generano la sottostringa $w_i w_{i+1} \dots w_j$. Per $i > j$, le voci della tabella non sono utilizzate. L'algoritmo riempie le voci della tabella per ogni sottostringa di w . In primo luogo si riempiono le voci corrispondenti alle sottostringhe di lunghezza 1, poi quelle di lunghezza 2 e così via. Si utilizzano le voci per le lunghezze minori per determinare quelle per le lunghezze superiori.

Per esempio, si supponga che l'algoritmo abbia già stabilito quali variabili generano tutte le sottostringhe di lunghezza fino a k . Per determinare se una variabile A genera una particolare sottostringa di lunghezza $k + 1$, l'algoritmo suddivide la sottostringa in due parti non vuote in tutti i k modi possibili. Per ogni suddivisione, l'algoritmo esamina ogni regola $A \rightarrow BC$ per determinare se B genera la prima parte e C genera la seconda parte,

utilizzando le voci della tabella precedentemente calcolate. Se entrambe B e C generano le rispettive parti, A genera la sottostringa e quindi viene aggiunta la voce corrispondente nella tabella. L'algoritmo inizia il processo con le stringhe di lunghezza 1 esaminando la tabella per le regole $A \rightarrow b$.

DIMOSTRAZIONE. Il seguente algoritmo D implementa l'idea della dimostrazione. Sia G una CFG in forma normale di Chomsky che genera il CFL L . Si supponga che S sia la variabile iniziale. (Ricordiamo che la stringa vuota viene gestita in modo speciale nella grammatica in forma normale di Chomsky. L'algoritmo gestisce il caso particolare in cui $w = \epsilon$ nella fase 1.) I commenti appaiono tra doppie parentesi quadre.

```

D = "Su input  $w = w_1 \dots w_n$ :
1. Per  $w = \epsilon$ , se  $S \rightarrow \epsilon$  è una regola, accetta; altrimenti, rifiuta.
   [caso  $w = \epsilon$ ]
2. Per  $i = 1$  to  $n$ : [esamina ogni sottostringa di lunghezza 1]
3. Per ogni variabile  $A$ :
4. Testa se  $A \rightarrow b$  è una regola, dove  $b = w_i$ .
5. Se sì, poni  $A$  in  $table(i, i)$ .
6. Per  $i = 2$  to  $n$ : [i è la lunghezza della sottostringa]
7. Per  $i = 1$  to  $n - i + 1$ : [i è la posizione iniziale della sottostringa]
8. Sia  $j = i + i - 1$ . [j è la posizione finale della sottostringa]
9. Per  $k = i$  to  $j - 1$ : [k è la posizione di separazione]
10. Per ogni regola  $A \rightarrow BC$ :
11. Se  $table(i, k)$  contiene  $B$  e  $table(k + 1, j)$  contiene  $C$ , pone  $A$  in  $table(i, j)$ .
12. Se  $S$  è in  $table(1, n)$ , accetta; altrimenti, rifiuta."
```

Ora analizziamo D . Ogni fase è facilmente implementata in modo da essere eseguita in tempo polinomiale. Le fasi 4 e 5 sono eseguite al più nv volte, dove v è il numero di variabili in G ed è una costante fissa indipendente da n ; quindi queste fasi sono eseguite $O(n)$ volte. La fase 6 è eseguita al più n volte. Ogni volta la fase 6 viene eseguita, la fase 7 viene eseguita al più n volte. Ogni volta che la fase 7 viene eseguita, le fasi 8 e 9 sono eseguite al più n volte. Ogni volta che la fase 9 viene eseguita, la fase 10 è eseguita r volte, dove r è il numero di regole di G ed è un'altra costante. Quindi la fase 11, il ciclo interno dell'algoritmo, viene eseguito $O(n^3)$ volte. La somma totale mostra che D esegue $O(n^3)$ fasi.

TEOREMA

7.43 Si mostri che NP è chiusa rispetto all'operazione star.

DIMOSTRAZIONE

7.43 Sia $A \in \text{NP}$. Si costruisce una NTM M per decidere A^* in tempo polinomiale non deterministico.

$M =$ "Su input w :

1. Dividi non deterministicamente w in parti $w = x_1 x_2 \dots x_k$.
2. Per ciascun x_i , ipotizza non deterministicamente il certificato che mostra che $x_i \in A$.
3. Verifica tutti i certificati se possibile, poi accetta. Altrimenti, se la verifica fallisce, rifiuta."

TEOREMA

7.37 Si mostri che se $P = NP$, esiste un algoritmo di tempo polinomiale che prende un grafo non orientato in input e trova una clique massima contenuta nel grafo. (Si veda la nota nel Problema 7.35.)

7.37 Assumendo che $P = NP$, risulta $CLIQUE \in P$, ed è possibile verificare se G contiene una clique di dimensione k in tempo polinomiale, per ogni valore di k . Verificando se G contiene una clique per ciascuna dimensione, da 1 fino al numero di nodi di G , è possibile stabilire la dimensione t di una clique massimale di G in tempo polinomiale. Una volta nota t , è possibile trovare una clique con t nodi come segue. Per ciascun nodo x di G , si rimuove x e si calcola la dimensione massima risultante in una clique. Se la dimensione risultante decresce, si rimette x e si continua con il nodo successivo. Se la dimensione risultante è ancora t , si rimuove x permanentemente e si continua con il nodo successivo. Quando tutti i nodi sono stati considerati in questo modo, i nodi restanti costituiscono una clique di dimensione t .

TEOREMA

7.50 Sia $HALF-CLIQUE = \{ \langle G \rangle \mid G \text{ è un grafo non orientato che possiede un sottografo completo con almeno } m/2 \text{ nodi, dove } m \text{ è il numero di nodi di } G \}$. Si mostri che $HALF-CLIQUE$ è NP-completo.

DIMOSTRAZIONE

7.50 Diamo una funzione di riduzione di tempo polinomiale da $CLIQUE$ a $HALF-CLIQUE$. L'input della riduzione è la coppia $\langle G, k \rangle$ e la riduzione produce il grafo $\langle H \rangle$ in output dove H è come segue. Se G ha m nodi e $k = m/2$, allora $H = G$. Se $k < m/2$, allora H è il grafo ottenuto da G aggiungendo j nodi, ciascuno connesso ad ognuno dei nodi originali e tra loro, dove $j = m - 2k$. Pertanto, H ha

$m + j = 2m - 2k$ nodi. Si noti che G ha una clique di dimensione k se e solo se H ha una clique di dimensione $k + j = m - k$, e quindi $\langle G, k \rangle \in CLIQUE$ se e solo se $\langle H \rangle \in HALF-CLIQUE$. Se $k > m/2$, allora H è il grafo ottenuto aggiungendo j nodi a G senza archi aggiuntivi, dove $j = 2k - m$. Quindi, H ha $m + j = 2k$ nodi, e pertanto G ha una clique di dimensione k se e solo se H ha una clique di dimensione k . Per cui, $\langle G, k \rangle \in CLIQUE$ se e solo se $\langle H \rangle \in HALF-CLIQUE$. Occorre anche far vedere che $HALF-CLIQUE \in NP$. Il certificato è semplicemente la clique.

TEOREMA 5.22

Se $A \leq_m B$ e B è decidibile, allora A è decidibile.

DIMOSTRAZIONE. Siano M il decisore per B ed f la riduzione da A a B . Descriviamo un decisore N per A come segue.

$N =$ “Su input w :

1. Computa $f(w)$.
2. Esegue M su input $f(w)$ e restituisce lo stesso output di M .”

Ovviamente, se $w \in A$, allora $f(w) \in B$ perché f è una riduzione da A a B . Quindi, M accetta $f(w)$ ogni volta che $w \in A$. Quindi, N svolge il compito desiderato.

Continua pag 251